Information Security

The Go-Deeper Guide: Lectures 1 to 5, Quiz 1 and the six TPs

Pr. Imane Fouad's course, UM6P. Every section: a compact recap of the slides, then the details and hard questions a demanding examiner asks.

How to use this guide

You already know the basics, so each section opens with a short *Core* box (the slide content, compressed) and then spends its effort on the *Go deeper* rubric: the exact numbers, dates, names, edge cases and mechanisms that separate a pass from a top mark, followed by *Hard questions, answered* written in the style of the professor's own Quiz 1 (multi-select, scenario-based, "which statements are correct", "why does this happen", "is this sufficient"). Appendix A solves Quiz 1 question by question with the reasoning; Appendix B lists what each TP teaches that can come back on an exam. Read a section's Core, then test yourself on the hard questions *before* reading the answers.

Contents

1 Lecture 1: Foundations **3**

1.1 Precision of thought: "is this circle secure?" . . . 3
1.2 Information, security, and the CIA triad . . . 3
1.3 Risk, threats, vulnerabilities, threat models . . . 4
1.4 Security services and controls . . . 5
1.5 Threat actors and attack types . . . 6
1.6 Malware taxonomy and infection vectors . . . 7
1.7 Areas of information security and course structure . . . 9

2 Lecture 2: Stateful tracking **9**

2.1 What the Internet can know, and what privacy is . . . 9
2.2 Web fundamentals: URL, HTTP, HTML, browser model . . . 10
2.3 Sessions: identifier in the URL vs cookies . . . 11
2.4 Cookie directives . . . 11
2.5 How web tracking works; first party vs third party . . . 12
2.6 Cross-site tracking, analytics, cookie respawning . . . 13
2.7 Cookie syncing and real-time bidding . . . 15
2.8 Clickjacking (TP1) and framing defences . . . 16

3 Lecture 2b: Stateless tracking, protections **17**

3.1 Web evolution and the attack surface . . . 17
3.2 Browser fingerprinting: definition, stateful vs stateless . . . 17
3.3 Specific attributes: Battery, User-Agent, canvas, GPU, mobile, IP . . . 18
3.4 Properties: uniqueness, linkability, consistency; entropy . . . 19
3.5 Fingerprinting for security . . . 20

3.6 Protection: filter lists and behaviour-based blocking . . . 21
3.7 Browsers against tracking: third-party cookies, Privacy Sandbox, isolation, randomisation . . . 22

4 Lecture 3: Classical and symmetric crypto 23
4.1 Terminology and goals . . . 23
4.2 Diffie–Hellman and the Merkle story . . . 24
4.3 Classical ciphers and how to break them . . . 25
4.4 XOR and the attack models . . . 26
4.5 Block cipher modes of operation . . . 27
4.6 Symmetric ciphers in use; DES . . . 28
4.7 AES . . . 29

5 Lecture 5: Asymmetric, hashing, PKI, stego 31
5.1 Why asymmetric: key management and the mailbox idea . . . 31
5.2 RSA . . . 32
5.3 Man in the middle: confidentiality is not enough . . . 33
5.4 Hash functions . . . 33
5.5 Authentication: MACs and digital signatures . . . 34
5.6 Certificates, chain of trust, PKI . . . 35
5.7 Steganography . . . 37

A Quiz 1 (Web Privacy, “The Alice Privacy Investigation”), solved 37

B TP takeaways 38

1 Lecture 1: Foundations of Information Security

1.1 Precision of thought: "is this circle secure?"

Core (what the slides say)

The opening lesson: the question is under-defined. "Secure" means nothing until you say *against what* ("if I flood-fill outside the circle, will the colour penetrate it?"). Security reasoning starts by defining the goal; shortcuts in this definition are the root of real failures. Statistics quoted: ≈ 90% of cyber attacks involve human error or behaviour; average cost of a data breach $3.86 million; average time to identify a breach 191 days.

Go deeper: details she can ask about

- The "misconceptions quiz" answers: a link is read *right to left* like a postal address, so `http://facebook.mobile.com` goes to the domain `mobile.com` (the registrable domain is the last two labels; `facebook` is just a subdomain name chosen by whoever owns `mobile.com`). Google *can* know what you typed before you press Enter (autocomplete sends keystrokes). Google *cannot* see your desktop background (a web page is sandboxed; it sees browser-exposed properties only, which is exactly what Lecture 2b is about). Links in an email are not safe just because the visible text looks right; inspect the real target.
- The "precision" principle returns in every later definition: a threat model must name the asset, the assumed vulnerability and the attacker's capabilities before you can claim a defence works.

Hard questions, answered

Q. Why does "90% of attacks are caused by human error" matter for the design of controls?
A. Because it shifts weight from purely technical controls to administrative ones (awareness training, procedures, chain-of-command rules against acting on a random email) and to designs that fail safely when a human makes a mistake (MFA so a phished password alone is not enough).

Q. `http://facebook.mobile.com` vs `http://mobile.facebook.com`: which belongs to Facebook?
A. Only the second. The registrable domain is read from the right: `facebook.com` vs `mobile.com`. The same reading is used in Lecture 2 to decide first- versus third-party (eTLD+1).

1.2 Information, security, and the CIA triad

Core (what the slides say)

ISO 27000/27002: *information is an asset that has value to an organisation and needs to be suitably protected*, whatever its form (digital files, printed documents, emails, verbal communication, knowledge in people's heads). Security in general: protection of assets against threats. ISO 27001: *information security is the preservation of confidentiality, integrity and availability of information; other properties such as authenticity, accountability, non-repudiation and reliability can also be involved.*

- **Confidentiality**: information is not made available or disclosed to unauthorised individuals, entities or processes ("data is not leaked").
- **Integrity**: data or resources are not improperly changed, deleted or corrupted. *Data integrity*: not altered or destroyed in an unauthorised manner. *System integrity*: safeguarding the accuracy and completeness of assets.
- **Availability**: accessible and usable upon demand by an authorised entity.

- Also: **Accountability** (actions traceable to those responsible), **Authentication** (user or data origin accurately identifiable).

Examples: the airport boarding pass that is not checked against the record (Eve flies under Alice's name, no record of Eve flying): the pass *lacks integrity*. Targeted advertisement on Alice's private browsing data: *lacks confidentiality*. The submission platform crashing before the deadline: *lacks availability*.

Go deeper: details she can ask about

- She distinguishes *data* integrity from *system* integrity; a buffer overflow that injects code into a trusted server process breaks *system* integrity even if no stored record changed.
- A security *service* (goal, property) is *what* we want; a *control* (mechanism) is *how* we get it. "Encryption" is never a goal; "confidentiality" is never a mechanism.
- "Security is everywhere": physical (locks, guards, badges), digital (passwords, firewalls, encryption), human (awareness, behaviour). Security is a human and societal problem, not only a technology problem.
- The airport example is subtle: the *person–pass* check (name matches pass) is offline and the *pass–record* check happens at the gate; the missing link is pass authenticity. Eve bought a ticket in Alice's name with a stolen card and forged a pass in her own name; nothing ties the pass to the record once that check is skipped.

Hard questions, answered

Q. Classify each step of Eve's attack on Acme (the slide exercise).
A. (1) Phishing yields a valid username and password: *confidentiality* of credentials is broken and, consequently, *authentication* is undermined (Eve can now authenticate as the employee). (2) Buffer overflow inserts code into a trusted server process running as root: *integrity* (system integrity) is broken; arguably authentication too, since code runs under an identity Eve does not own. (3) Disabling the database logging process: *accountability* (actions are no longer traceable). (4) Creating a new database user by modifying the user table: *integrity* of the authorisation data and *authentication* (a fake identity now exists). (5) Flooding the web server with many computers: *availability*; copying the database meanwhile: *confidentiality*.

Q. Why is "accountability" listed separately from the triad, and which control supports it?
A. Because you can preserve C, I and A and still be unable to say *who* did what. Logging and auditing (administrative/technical detective controls) support it, which is why attackers disable logs first.

Q. An unauthorised employee reads a medical record without changing it. Which property?
A. Confidentiality only (Quiz 1, Q1). Changing a lab result while it stays accessible: integrity (Q2). A DoS on the portal: availability (Q3).

Q. Give a scenario where availability is broken without any attacker.
A. The ISO definition says "accessible upon demand by an authorised entity"; a crash or an expired certificate before a deadline breaks availability with no adversary, which is why corrective controls (backups, restore from a known-good image) exist.

1.3 Risk, threats, vulnerabilities, threat models

Core (what the slides say)

Jim Anderson: "information security is a well-informed sense of assurance that the information risks and controls are in balance." **Risk**: the potential that a threat will exploit a vulnerability and cause harm to an asset. **Threat**: something that can cause damage. **Vulnerability**: a

weakness that can be exploited. **Asset**: what we want to protect (data, systems, people). A **threat model** defines (i) the asset(s) at risk, (ii) the type of vulnerability you assume exists and protect against, (iii) the attacker's capabilities and knowledge; only then can you say how a defence prevents, detects, responds to or recovers from the attack. HTTP example: asset = private communications incl. credentials; vulnerability = packets intercepted in transit (open Wi-Fi); attacker = knows when/how to intercept packets of a user or of the site; control = HTTPS.

Go deeper: details she can ask about

- "Risks and controls in balance" is the key phrase: more control is not automatically better (cost, usability); security is a *risk-management* discipline (GRC: identify risks, assess likelihood and impact, *treat / accept / transfer / avoid*, monitor).
- The four responses in the threat-model slide are the functional roles of controls: *prevent, detect, respond, recover*.
- Adversaries are concrete: behavioural advertisers (Example 1), Guccifer 2.0 (stole and leaked DNC documents), and even the NSA has adversaries (Snowden). The point: "who is the adversary" is part of the model; the same system is secure against one and not another.

Hard questions, answered

Q. Write the threat model for a password database stolen offline.
A. Asset: user passwords (and the accounts they protect). Vulnerability: the attacker has obtained the stored verifiers. Attacker capability: unlimited offline guessing at machine speed. Defence: hashing with per-user salt and a slow password hash (Lecture 5) so each guess costs and precomputed dictionaries do not transfer across users.

Q. Is HTTPS a complete defence in the threat model given on the slides?
A. Within that model, yes: it defeats passive interception in transit. It does not address a different vulnerability (a phished password, a malicious extension, a compromised endpoint) because those are outside the stated model; changing the attacker's capabilities changes the answer.

Q. Difference between a threat and a vulnerability in one sentence each, with the HTTP case.
A. Threat: an eavesdropper on open Wi-Fi (the agent that can cause harm). Vulnerability: HTTP sends packets in clear (the weakness that can be exploited). Risk: the eavesdropper reading credentials.

1.4 Security services and controls

Core (what the slides say)

Services = what we want (goals/properties: C, I, A...); controls = how we achieve them (mechanisms, actions, tools, procedures: encryption, firewalls, awareness). Control *implementation* categories: **physical** (facility protection, guards, locks, monitoring, environmental controls, intrusion detection), **technical** (logical access control, cryptographic controls, security devices, user authentication, intrusion detection, forensics), **administrative** (policies, standards, procedures and practice, personnel screening, awareness training). Control *functional* types: **preventive** (prevent attempts to exploit vulnerabilities; e.g. file encryption), **detective** (warn of attempts; e.g. IDS), **corrective** (correct errors that have been detected; e.g. restoring from the last known-good image). Speed round: MFA = technical/preventive; firewall = technical/preventive; cameras = physical/detective; phishing training = administrative/preventive; backups restored after ransomware = technical/corrective; IDS alerts = technical/detective.

Go deeper: details she can ask about

- Every control has *two* labels (implementation type *and* functional type); the exam asks for both.
- "Intrusion detection" appears under both physical and technical: a motion sensor in a server room is physical/detective; a network IDS is technical/detective.
- Backups themselves are preventive-ish (they prepare recovery), but *restoring* them is corrective; the slide classifies by the action described.
- A control can be the wrong tool for a goal: encryption gives confidentiality, not availability (ransomware is encryption used *against* availability).

Hard questions, answered

Q. Classify: a security guard checks badges at the entrance; a log review each morning; an automatic account lockout after 5 failed logins; a policy requiring password rotation.
A. Guard checking badges: physical/preventive. Morning log review: administrative (a procedure)/detective. Account lockout: technical/preventive (some say deterrent; stay with preventive). Password-rotation policy: administrative/preventive.

Q. Name one control that supports accountability and one that supports authentication, each with both labels.
A. Accountability: audit logging (technical/detective). Authentication: MFA (technical/preventive).

Q. Can one control be preventive and detective at the same time?
A. Yes; a firewall that blocks and logs is preventive (blocking) and provides detective data (logs). Classify by the function being discussed.

1.5 Threat actors and attack types

Core (what the slides say)

A **hacker** is not necessarily malicious (passion and ability to bypass systems). **Black hat**: malicious, breaks the law for gain or sells services on the dark web. **White hat** (ethical hacker): sells services to find and fix vulnerabilities. **Grey hat**: typically a former black hat working with organisations but not necessarily respecting the law. Actors: **state-related** (very high resources, espionage/intrusion into critical systems), **cybercriminals** (high resources, financial gain), **cyber-terrorists** (destruction), **hacktivists** (political ideology, defacements with a message), **script kiddies** (low capability, use others' tools, for amusement). Attacks: social engineering (psychological manipulation), DDoS (previously compromised machines = *bots*, hours to days), malware infection, vulnerability exploits (OWASP Top 10; Metasploit in Kali), unauthorised access (social engineering, brute force, password cracking), privilege abuse (internal; possible when the insider has unnecessary privileges or little is logged), ransomware (encrypts data, demands ransom for the key). Password cracking: passwords are stored hashed; the attacker who has stolen the hashes tests candidate passwords *offline*.

Go deeper: details she can ask about

- The distinction the professor insists on: "hacker" is a skill description; the *hat* is the intent/legality. Grey hat is defined by cooperation *without* necessarily respecting the law, not by "sometimes good, sometimes bad".
- DDoS vocabulary: *bots*, *botnet*; the word "distributed" means the traffic comes from many compromised machines, which is what makes blocking by source address ineffective.
- Privilege abuse has two enabling conditions on the slide: unnecessary privileges for the role

(violates least privilege) *and* little or no logging (no accountability).
- Password cracking "assumes the attacker has already stolen the hash": the attack is offline, so rate limiting on the login page does not help; only the cost per guess (hash design) and the password's entropy do.

Hard questions, answered

Q. Why is a DDoS harder to stop than a DoS from one machine?
A. Traffic comes from many bots with legitimate-looking addresses; blocking one source does nothing and distinguishing bots from real users at the edge is hard; the control is capacity/absorption and upstream filtering, not a single block rule.

Q. A SOC analyst with read access to all logs exports customer data. Which attack type, which failed principle, which control?
A. Privilege abuse (insider); least privilege was violated (unnecessary access for the role); controls: role-based access, logging of exports (accountability), separation of duties.

Q. Why does the slide say "offline" for password cracking, and what does that change?
A. The hash file is in the attacker's hands; guesses are tested locally at full speed with no lockout. The defence moves from the login system to how the verifier is computed (salt, slow hash) and to password strength.

1.6 Malware taxonomy and infection vectors

Core (what the slides say)

Vectors sorted from *more technical* to *more social*: virus, macro virus, exploit, worm … drive-by download, clickjacking, phishing, spear phishing, trojan horse. **Virus**: infects *programs*; modifies them to include a copy of itself; executes secretly when the host runs; can do anything the host may do. Components: *infection mechanism* (vector), *trigger* (event/condition activating the payload; a *logic bomb*), *payload* (what it does besides spreading). Modern era: viruses modify binaries, so track hashes of binaries and stop unprivileged users modifying them; classical viruses "aren't really a thing" any more (the press still says "virus"). **Macro virus**: document scripting (Office); harder now (warnings, separate `.xlsm` extension). *Melissa* (1999): Word macro, accessed Outlook, mailed itself to contacts (virus = host + execution). **Worm**: seeks out *machines* to infect; each infected machine is a launching pad; spreads via network (web-app upload bug, SSH dictionary attack), shared media (USB autostart), or social techniques. Strategies: *random* scanning (high traffic), *hit-list* (precompiled targets, stealthy, e.g. Stuxnet), *topological* (info on the victim), *local subnet* (behind NAT/firewall). *Morris worm* (1988): first major Internet worm, experiment got out of hand, dictionary-attacked weak passwords, exploited `fingerd` and `sendmail`, first conviction under the 1986 Computer Fraud and Abuse Act. **Rootkit**: a program that uses root privilege to modify the running OS's behaviour (kernel code/data, system calls) so the OS *lies* (e.g. `ps` hides a process); *not* an infection method but a method of *stealth and continued access* (back door); requires root obtained by another attack. **Drive-by download**: browser vulnerabilities install malware when viewing an attacker-controlled page, automatically and invisibly (Angler exploit kit, mid-2010s: visit → redirect → check for vulnerability → exploit → install). **Clickjacking**: the user gives input for one action but, by timing or circumstance, performs another (download dialog appears under the "puppy" 200 ms later; a transparent authorisation page over an unrelated interface). **Phishing**: attacker pretends to be a trusted source and induces an action (click, open attachment, reply with info, change a setting, transfer money); *spear phishing*: targeted, deeper narrative, leverages facts from other investigation. Google Docs attack (2017): a fake share invitation led to a malicious app authorisation request; granting permission exposed the account; it then spread to contacts (phishing need not install malware).

Defences: users skeptical, mind what you make public, inspect links, go to known services yourself; organisations train users, clear chain-of-command procedures, email link screening. **Trojan horse**: malware the victim runs themselves (`adorablecat.jpg.exe`, a fake ad, "install MalMeeting plugin"); Zeus/Zbot banking trojan. Summary: viruses infect executables to spread; worms infect machines to spread; trojans infect machines if you run them; rootkits infect kernels to avoid detection/removal.

Go deeper: details she can ask about

- The axis of the vector diagram is *technical engineering vs social engineering*; an exam can ask you to place an attack on it and justify (drive-by: technical, no user action beyond visiting; trojan: social, the user runs it).
- A rootkit is the trick answer: it is *not* a vector. "Runs at boot" does not imply rootkit; a rootkit must alter OS behaviour. It implies you already have root.
- Virus vs worm is *what is infected* (program vs machine) and *who triggers execution* (host program runs vs self-propagation).
- Ransomware is a *payload* category, not a vector; the same sample can be worm (vector) + ransomware (payload).
- "Guess the malware type": Scenario A (infection through an outdated Windows service, automatic LAN scanning, 200,000 machines in 150 countries, files encrypted, Bitcoin ransom) describes a *worm* with a *ransomware* payload (WannaCry-like). Scenario B (compromised accounting-software update, spreads by network exploit *and* stolen credentials, ransom note but paying never recovers files) is a *supply-chain* delivered *worm* whose payload is a *wiper* disguised as ransomware (NotPetya-like); the lesson is that the apparent category (ransomware) and the real goal (destruction) differ.
- Phishing defence detail: "email link screening lets you see all outgoing clicks, spot trends and black-hole links at the forwarding stage": a *detective* control that becomes *corrective*.
- Clickjacking is both in Lecture 1 and TP1; the defences (`X-Frame-Options`, CSP `frame-ancestors`) are in Appendix B.

Hard questions, answered

Q. Why are classical viruses rare today, and what control made them so?
A. Their infection strategy is distinctive: they modify binary executables. Preventing unprivileged users from modifying binaries and tracking binary hashes (integrity monitoring, code signing) detects the modification, so the strategy stopped paying.

Q. A program executes at every boot and hides its files from Explorer. Rootkit?
A. Hiding files by changing what the OS reports is rootkit behaviour (the OS lies). Running at boot alone is not. To be a rootkit it must run with root privilege and alter kernel code or data; otherwise it is ordinary persistent malware.

Q. Which worm strategy would a stealthy, targeted attacker choose, and why not random scanning?
A. Hit-list: the worm carries a precompiled list of vulnerable targets, so little noisy scanning is needed (Stuxnet is the slide's example). Random scanning generates high traffic that may be inadvertently disruptive and reveals the worm.

Q. The 2017 Google Docs attack installed no malware. Why is it still phishing, and what did the attacker obtain?
A. Phishing attacks the human through deception to obtain an action; here the action was granting OAuth permissions to a malicious app, which exposed account access and let the attack spread to contacts. Credentials or authorisation are the goal, not necessarily code execution.

Q. Classify: a USB stick that writes an autostart executable to any attached drive.

A. Worm spreading through shared media (the slide's own example); if the executable only runs when the user opens it, the social component makes it closer to a trojan.

1.7 Areas of information security and course structure

Core (what the slides say)

Technical domains: network security (networks, traffic, firewalls), web security (applications, sessions, input), cryptography (encryption, hashing, signatures, PKI), access control (authentication, authorisation, least privilege), endpoint security (operating systems), cloud security (monitoring, detection, response). Governance domains: GRC (governance, risk and compliance), privacy (personal data protection). Course roadmap: Foundations (principles; threats and attacks) → technical domains → governance and people. Access control core concepts: **identification** (claiming an identity), **authentication** (proving it), **authorisation** (what the authenticated identity may do), **accountability** (logging/auditing). GRC: governance (direction and oversight, policies, roles, strategy), risk (identify, assess likelihood and impact, treat/accept/transfer/avoid, monitor), compliance (laws, standards, contracts, evidence and audits). Cryptography's goals (preview): data privacy, integrity, authenticity. Web application overview: `subdomain.mysite.com/folder/page?id=5`; GET for pages and resources, POST for forms and logins.

Go deeper: details she can ask about

- IAAA order matters and each step is a different question: who are you claiming to be / prove it / what may you do / what did you do.
- Risk *treatment* options are exactly four: treat (mitigate), accept, transfer (insurance, outsourcing), avoid (stop the activity).
- References she cites: Whitman & Mattord; Stallings & Brown; Anderson's *Security Engineering*; ISO/IEC 27001, 27002; NIST; OWASP.

Hard questions, answered

Q. A badge reader opens a door for anyone holding a valid badge. Which of IAAA is missing?
A. Authentication of the *person* (the badge identifies; it does not prove the holder is the owner); MFA (badge + PIN) adds it.

Q. Buying cyber-insurance is which risk treatment? Deciding not to store card numbers at all?
A. Transfer; avoid.

2 Lecture 2: Web Privacy I, Stateful Tracking

2.1 What the Internet can know, and what privacy is

Core (what the slides say)

A website can know what you read, click, scroll or focus on; what interests you; which other sites you visited; who you are even without logging in; where you are or regularly go; your location from nearby Wi-Fi networks. Cases: Cambridge Analytica (Wylie, 18 March 2018: harvested millions of Facebook profiles, built models to "target their inner demons"); Google paid $392 million to 40 US states (November 2022) over misleading location tracking. **Privacy** concerns how information about people is collected, used, shared and retained, and a person's ability to

understand what happens to their information, influence how others use it, and avoid unwanted observation or interference. Your Google data: myactivity, ads settings, permissions, YouTube history, Takeout.

Go deeper: details she can ask about

- The privacy definition has *two halves*: the data life-cycle (collect, use, share, retain) and the person's *agency* (understand, influence, avoid). "Targeting attributes can be invasive" is the bridge to the tracking ecosystem.
- Course objectives list the exam scope exactly: web basics; explain stateful tracking; explain stateless tracking; identify techniques; understand risks; explore protections; evaluate their limitations.

2.2 Web fundamentals: URL, HTTP, HTML, browser model

Core (what the slides say)

History: Bush's *Memex* (1945, non-linear navigation); Berners-Lee's *ENQUIRE* (1980, CERN, local only); the 1989 proposal *Information Management: A Proposal* that became the WWW. **URL** identifies and locates resources (DNS maps names to IPs): `https://informationsecurity.com:443/courses/web-security?week=5&topic=xss` = protocol, hostname, port, path, query string. **HTTP**: application-layer, client–server, request–response, *stateless* (the server keeps no state after the TCP connection closes); request = URL + method (GET, POST…) + headers + optional body; response = status code + resource or error. **HTML**: declarative markup (structure, not logic). Browser model: load content, render (parse HTML, run scripts/plugins), respond to events (OnClick, OnMouseover, OnLoad, OnBeforeUnload, onerror, setTimeout). Client stores cookies and session info and runs JavaScript; server runs business logic, authenticates, enforces access control, talks to the database. The `location` object: `href` (full URL incl. fragment), `host` (host + port), `hostname`, `port`, `protocol`, `pathname`, `search` (query).

Go deeper: details she can ask about

- "Stateless" is the reason sessions and therefore cookies exist; it is also why tracking is a *design problem* rather than a bug: the web needs state and trackers reuse the mechanism.
- `location.host` includes the port, `location.hostname` does not; `location.href` includes the fragment (`#...`), which is never sent to the server.
- Static sites never needed state; "applications" (shopping cart) do.

Hard questions, answered

Q. Which part of a URL is never transmitted to the server, and why does that matter for tracking?
A. The fragment after `#`. A tracker that wants context must read it with JavaScript (`location.href`) and send it explicitly; nothing in HTTP carries it.

Q. Why does "HTTP is stateless" not mean "HTTP is anonymous"?
A. Each request still carries the IP address and headers (User-Agent, Accept-Language…) that distinguish browsers passively (Lecture 2b); statelessness only means the server does not *remember*; it can still *recognise*.

2.3 Sessions: identifier in the URL vs cookies

Core (what the slides say)

Option 1, session ID in every link (`?sessionid=8F4A92C1`): high accidental leakage; the URL is transmitted in the `Referer` header, so the session leaks to every included third-party site. Option 2, **cookies** (1994): a random token sent with `Set-Cookie`; the client sends it back on *every* request to that server, *regardless of the initiating site*; persisted in the browser; expiry controlled by `Expires`, default deleted at session end (browser closed); ending the session = deleting the cookie.

Go deeper: details she can ask about

- The phrase "regardless of initiating site" is the seed of third-party tracking and of CSRF; `SameSite` was invented decades later to restrict exactly this.
- The `Referer` leak is still real today for URL parameters; this is why identifiers belong in cookies or headers, not in URLs, and why cookie syncing (which deliberately puts an ID in a URL) is a privacy problem.

Hard questions, answered

Q. Give two different ways a session ID in a URL leaks and one way a cookie does not.
A. Leaks: the Referer header sent to third parties embedded in the page; copy-pasted or shared links; browser history and server logs. A cookie is not part of the URL, so sharing the link does not share the session; it is only sent to its own domain.

2.4 Cookie directives

Core (what the slides say)

`<name>=<value>`; `Expires=<date>` or `Max-Age=<seconds>` (when to delete); `Domain=<domain>`: defaults to the current host *without subdomains*; may be set to a parent domain, then applies to it and all its subdomains (`Domain=example.com` set from `websec.example.com` applies to `*.example.com` and `example.com`); `Path=<path>`: only this path and sub-paths. `HttpOnly`: no access from JavaScript via `document.cookie`. `Secure`: only transmitted over a secure connection; can only be set from HTTPS. `SameSite=Strict|Lax|None`: Strict = never sent on cross-site requests; Lax = sent only on "safe" top-level navigation (safe methods per RFC 7231: GET, HEAD, OPTIONS, TRACE); None = explicit opt-in to cross-site requests, *requires Secure*; browsers default to Lax. Example: `Set-Cookie: test=1; Domain=.example.com; Secure; HttpOnly; SameSite=None` is sent on any HTTPS request to example.com and subdomains, not readable by JS, sent cross-site, deleted on browser close (no expiry). The same without `Secure`: Chrome refuses it; Firefox and Edge warn. *Session cookie*: no long-term expiry, removed when the session ends (login state, carts). *Persistent cookie*: has Expires/Max-Age, survives restarts.

Go deeper: details she can ask about

- Lax has *two* conditions: top-level navigation (the address bar changes) *and* a safe method. A cross-site `<img>` or `<iframe>` load is not top-level, so Lax cookies are not sent; a cross-site POST form is top-level but not safe, so not sent either. Only a link click (GET) from another site carries Lax cookies.
- `Secure` has two effects: send only over HTTPS *and* can only be *set* from HTTPS. `SameSite=None` without `Secure` is rejected by Chrome.
- `HttpOnly` protects against reading via `document.cookie` (XSS theft) but the cookie is still

sent automatically; it does nothing against CSRF-style use. SameSite is the CSRF/tracking control; HttpOnly is the script-theft control.
- `Domain` semantics: setting a cookie for a parent domain widens the scope; you cannot set it for an unrelated domain or a public suffix. Without `Domain`, subdomains do not receive it (host-only cookie).
- Expiry: no `Expires`/`Max-Age` = session cookie, even if other attributes are present (the slide example is a session cookie despite looking "permanent").

Hard questions, answered

Q. Quiz 1, Q4. Cookie `aid=A72; Domain=example.test; Path=/; Secure; HttpOnly; SameSite=Lax` set by `https://shop.example.test`. Which requests carry it?
A. (A) From `https://shop.example.test`, loading `https://cdn.example.test/banner.png`: same site (both under `example.test`), HTTPS, Domain covers subdomains: *yes*. (B) Same over `http://`: `Secure` blocks: *no*. (C) From `https://social.test`, Alice *clicks a link* to `https://shop.example.test/profile`: cross-site but top-level GET navigation, Lax allows: *yes*. (D) `social.test` *embeds* `https://shop.example.test/tracker.png`: cross-site subresource, Lax blocks: *no*. Answer: A and C.

Q. A bank sets its session cookie `HttpOnly; Secure` but not SameSite. An attacker's page auto-submits a transfer form to the bank. Does the cookie go along?
A. With the modern default (Lax) a cross-site POST does not carry it. If the browser treated it as None (older behaviour), it would; HttpOnly and Secure are irrelevant to this question because they do not restrict automatic sending.

Q. Can `https://a.example.com` set a cookie that `https://b.example.com` receives? And one for `example.org`?
A. Yes with `Domain=example.com` (parent domain, applies to all subdomains). No for `example.org`: a different registrable domain.

Q. Why does a tracker set `SameSite=None; Secure` and a bank `SameSite=Strict`?
A. The tracker's cookie must be sent on requests embedded in *other* sites, which only None allows (and None requires Secure). The bank wants its cookie never attached to requests initiated elsewhere to block CSRF; Strict even blocks link clicks, which is why some sites use Lax for usability.

2.5 How web tracking works; first party vs third party

Core (what the slides say)

Mechanisms a tracker needs: (1) store/create an identity in the browser: HTTP cookies, HTTP headers, browser storages (*stateful*), or device fingerprinting from browser/OS properties and IP (*stateless*); (2) communicate the identity back: HTTP requests made by the browser, JavaScript; (3) associate the user with the current context: Referrer, location. **First-party** request: same registrable domain (**eTLD+1**) as the visited site; **third-party**: a different eTLD+1. Visiting `https://shop.example.com` (registrable domain `example.com`): `www.example.com`, `images.example.com`, `api.shop.example.com`, `analytics.example.com` are first party; `example.org`, `example.com.tracker.net`, `cdn.google.com`, `shop-example.com` are third party. The UM6P site exercise: 69 domains can learn that you visited. First-party cookie: belongs to the visited site; third-party cookie: belongs to another domain embedded in the page. Browser cookie database keyed by domain (`bbc.com: id=123`; `doubleclick.com: id=456`).

Go deeper: details she can ask about

- eTLD+1 = effective top-level domain plus one label. Effective TLDs include public suffixes like `co.uk`; so `bbc.co.uk` and `news.bbc.co.uk` are the same site. `example.com.tracker.net` reads from the right: `tracker.net`; it is third party no matter how much it resembles `example.com` (same lesson as `facebook.mobile.com`).
- `shop-example.com` is a different registrable domain: a hyphen makes a different name.
- "Site" (for SameSite) is eTLD+1; "origin" (for the same-origin policy and `document.cookie`) is scheme + host + port. Two subdomains are same-site but cross-origin.
- The three mechanisms map to the two lectures: storage (2: cookies/storages; 2b: fingerprint), transport (HTTP requests, JS), context (Referrer, location).

Hard questions, answered

Q. `analytics.example.com` embedded on `shop.example.com` is first party by eTLD+1. Can it still be a tracker?
A. Yes. First party is a *domain relationship*, not a statement of intent; CNAME cloaking (Lecture 2b) makes third-party trackers look first party precisely to exploit this.

Q. Why do trackers need all three mechanisms? Give the failure if one is missing.
A. No storage/recognition: every visit is a new user. No transport: the ID never reaches the tracker. No context: the tracker knows "user 123 came back" but not what they read; the Referrer (or a URL parameter the embedding script adds) supplies the page.

2.6 Cross-site tracking, analytics, cookie respawning

Core (what the slides say)

Cross-site tracking: based on third-party cookies. `bbc.com` embeds `<script src=doubleclick.net>`; doubleclick sets `id=1234`; later `nytimes.com` embeds the same script and the browser sends `id=1234`; doubleclick's log: `bbc.com 15:00, nytimes.com 16:00`. **Analytics** (within-site only): `google-analytics.com/script.js` runs in the publisher's context and sets a *first-party* cookie (`accuweather.com: ga_id=123`); it reports `track?ga_id=123&site=accuweather.com`; it cannot collect history across sites. **Respawning** (zombie cookies): the cookie comes back after deletion from HTML5 `localStorage` (across sessions only), Flash LSOs (across sessions *and* browsers), or HTTP headers `ETag`/`Last-Modified`. `localStorage`: key/value strings, no expiry, `setItem/getItem/removeItem`. ETag respawning: the ETag is a caching mechanism (`200 OK Etag: "x234dff"`; next request `If-None-Match: "x234dff"`; `304 Not Modified`), so an identifier placed in the ETag is echoed back even in private browsing with cookies blocked (KissMetrics lawsuit, August 2011; `_km_cid` cookie equal to the ETag). Evercookie (samy.pl) combines many storages. All of these storages can also be used for tracking *without* cookies at all. Large-scale studies: Roesner et al. (NSDI 2012) classified tracking behaviour; Englehardt & Narayanan (CCS 2016, OpenWPM, 1 million sites) measured third-party presence.

Go deeper: details she can ask about

- Analytics scripts run in the *publisher's* origin, so they read and write the publisher's cookies (unless HttpOnly) and the identifier differs per publisher; combined profiles across sites need an explicit link (same login, or the script sending something stable). Quiz 1 Q5/Q7 are built on this.
- The request that fetches `analytics.js` from `metrics.test` carries `metrics.test`'s cookies, never the publisher's; the publisher's cookie reaches the analytics server only if the script

reads it and sends it explicitly (in a URL parameter or body).
- Respawning flow (slides 63–65): tracker's iframe stores `id=789` in both the cookie and localStorage; after cookie deletion the server issues a new cookie `555`, but the script finds `789` in localStorage and reports `old_id=555, new_id=789`; the server merges the profiles. So the user sees a new cookie value and is still linked.
- Flash LSOs were shared across browsers on the same machine, which is why they could respawn across browsers; localStorage is per browser profile and per origin.
- ETag detail: the cache must still hold the resource; clearing the cache clears the ETag. Private browsing with cookies blocked did *not* clear it in the KissMetrics case because caching still worked.

Hard questions, answered

Q. Quiz 1, Q5. Shop stores non-HttpOnly first-party cookie `prefs`; the page loads `metrics.test/analytics.js` and an iframe from `tracker.test`.
A. (A) The analytics script executes in the shop's context and can read `prefs`: true. (B) Code inside the iframe can read `prefs`: false, the iframe is a different origin, `document.cookie` is per origin. (C) The request for `analytics.js` automatically sends `prefs` to `metrics.test`: false, cookies go to their own domain. (D) The script can read `prefs` and explicitly send it: true. Answer: A and D.

Q. Quiz 1, Q7. Third-party cookies blocked; `news.test` and `shop.test` both load the same script from `metrics.test`; the browser stores `aid=N31` on news and `aid=S84` on shop.
A. (A) They are first-party identifiers for their publishers: true. (B) The browser sends both to `metrics.test` automatically: false. (C) Same script implies same identifier: false (different origins, different cookie jars). (D) The script can read the current publisher's identifier and send it explicitly: true. Answer: A and D. Follow-up: can metrics.test build one cross-site profile? Only if it links N31 and S84 by other means (login, fingerprint); the script alone yields two profiles.

Q. Quiz 1, Q10/Q12. Tests: (2) delete cookie, cookie returns `1234`; (3) delete cookie + localStorage, still `1234`; (4) clear cookies, storage and cache, new values `5678`. Interpret and name the sufficient fix.
A. Test 2 is consistent with restoring from localStorage; it does not prove fingerprinting (several stateful stores could explain it). Test 3 (localStorage gone, cookie still back) is consistent with restoring from the cached ETag. Deleting the cookie did *not* stop the link to profile 3. Sufficient action: clear cookies, local storage and the relevant cache state *together* (Q12 A); deleting only cookies after each session (B), clearing storage but keeping the cache (C) or shortening expiry (D) leave a respawn source.

Q. Why can analytics using first-party cookies not reconstruct cross-site history, while a third-party cookie can?
A. The first-party cookie is scoped to the publisher's domain and is only sent to it; the analytics server receives a per-publisher identifier plus the page, so it sees "user 123 on accuweather" and "user 987 on nytimes" with no link. The third-party cookie belongs to the tracker's domain and is the *same* on every page that embeds the tracker.

2.7 Cookie syncing and real-time bidding

Core (what the slides say)

Cross-site tracking is *limited*: each tracker only sees the sites that embed it (doubleclick: bbc, nytimes; adnxs: bbc, jumia). **Cookie syncing**: the process by which two different trackers link the IDs they gave the same user ("user 123 for doubleclick is user 456 for quantserve"; one knows age/gender, the other email/address). Mechanism: a page includes tracker1, which responds with a redirect to tracker2 carrying tracker1's id in the URL; the browser follows it and sends tracker2's own cookie; tracker2 logs "user abc for tracker2 is user 123 for tracker1". **Targeted advertising** and **RTB** (real-time bidding): a protocol for transacting display ads in real time. Actors: **SSP** (supply-side platform: helps publishers offer and sell ad space, sends each opportunity to an ad exchange); **AdX** (ad exchange: runs the auction, distributes bid requests, selects the winner); **DSP** (demand-side platform: helps advertisers evaluate opportunities and bid automatically). In RTB, SSP `s.com` syncs its cookie with DSP `d.com` through an HTTP(S) redirect. RTB is based on personal data (cookies, browsing history, preferences); transparency is hard because only part of the process is visible in the browser and it is hard to establish what trackers collect. "If you're not paying for the product, you are the product" (Andrew Lewis).

Go deeper: details she can ask about

- The sync works because of two independent facts: the browser follows redirects automatically, and it attaches each domain's cookie to requests for that domain. Neither tracker ever reads the other's cookie (impossible across domains); the identifier crosses over in the *URL*.
- A bid request is *broadcast* to many DSPs; every DSP that receives it learns the page URL, approximate location and the pseudonymous ID, whether or not it wins (Quiz 1 Q9). The pseudonymous ID is stable, so it does *not* prevent linking future requests.
- "Hypothetical example" on the slide: the merged view (bbc 15:00, nytimes 16:00, jumia 17:00) only exists *after* syncing.

Hard questions, answered

Q. Quiz 1, Q16. Draw and explain cookie syncing between Tracker 1 (id 1234) and Tracker 2 (id 5678).
A. Browser visits a publisher that embeds Tracker 1 (pixel or script). Request to `tracker1.com/pixel` carries Tracker 1's cookie `id=1234`. Tracker 1 answers `302 Location: https://tracker2.com/sync?t1=1234`. The browser follows the redirect; the request to `tracker2.com` carries Tracker 2's own cookie `id=5678` and the URL parameter `t1=1234`. Tracker 2 records the pair (5678 ↔ 1234); optionally it redirects back to `tracker1.com/sync?t2=5678` so Tracker 1 stores the mapping too. From then on both can exchange data about this browser using either identifier, e.g. inside RTB bid requests.

Q. Quiz 1, Q8. `GET tracker-a.test/sync Cookie: aid=A42` → `302 Location: tracker-b.test/sync?aid=A42` → `GET tracker-b.test/sync?aid=A42 Cookie: bid=B71`.
A. (A) Tracker B can associate A42 with B71: true. (B) Tracker B receives A42 as one of its own cookies: false, it is a URL parameter. (C) Tracker A passes its identifier through the redirected URL: true. (D) Works only if both trackers share an origin: false, that is the whole point. Answer: A and C.

Q. Quiz 1, Q9. DSP-X receives a bid request and loses the auction.
A. (A) The SSP helps the publisher offer its ad space: true. (B) DSP-X may receive user information though it loses: true. (C) Only the winner can access the bid request: false. (D) The pseudonymous ID prevents linking future requests: false. Answer: A and B.

Q. Why is cookie syncing a bigger privacy problem than either tracker alone?

A. It merges partial profiles (demographics from one, contact data from the other) and extends each tracker's coverage to the union of the sites the other is embedded in; it also moves identifiers into URLs, where they leak via Referer and logs.

2.8 Clickjacking (TP1) and framing defences

Core (what the slides say)

TP1 builds the attack: a decoy game (`game.html`) embeds the legitimate Student Portal (`portal.html`) in an `<iframe>` positioned over the game with `opacity: 0` and a higher `z-index`; at a chosen score the visible "Catch Me" button is moved exactly under the hidden Login button, so Alice's click lands on the portal while she believes she is playing. Defence research: `X-Frame-Options` and Content Security Policy's `frame-ancestors` directive.

Go deeper: details she can ask about

- Why it works: the click goes to the *top-most* element at that point; a transparent iframe is still interactive. The attack needs the victim to be already authenticated (cookies sent with the framed page), which is also why `SameSite` cookies partially mitigate it (a Lax/Strict session cookie is not sent to a cross-site iframe, so the framed portal is not logged in).
- `X-Frame-Options: DENY` forbids any framing; `SAMEORIGIN` allows framing only by pages of the same origin; the old `ALLOW-FROM uri` is obsolete and ignored by modern browsers. It is a response header set by the *framed* site (the portal), enforced by the browser.
- CSP `frame-ancestors 'none'` (= DENY), `'self'` (= SAMEORIGIN), or a list of allowed origins (`frame-ancestors 'self' https://apps.um6p.ma`), which is the modern replacement and the only way to allow a *specific* trusted third site. When both headers are present, CSP `frame-ancestors` takes precedence.
- Answers to the TP's five questions: protections = X-Frame-Options, CSP frame-ancestors (plus JavaScript frame-busting, which is weak); each controls *who may embed this page*; same-origin only = `SAMEORIGIN` or `frame-ancestors 'self'`; for a portal that must never be framed, `frame-ancestors 'none'` (and `X-Frame-Options: DENY` for old browsers); if another trusted university application must embed it, switch to `frame-ancestors 'self' https://trusted.um6p.ma`, which X-Frame-Options cannot express.

Hard questions, answered

Q. Setting the game page's own headers to deny framing: does it protect Alice?
A. No. The header must be sent by the page that is *being framed* (the portal); the attacker controls the game page. Defences belong to the victim site.

Q. Does `opacity: 0` make the iframe non-interactive?
A. No; it is invisible but still receives events. `visibility: hidden` or `pointer-events: none` would stop interaction, which is why the attacker uses opacity.

Q. Which cookie attribute, set by the portal, would make the clickjacked login click harmless even without framing headers, and why is it not a complete defence?
A. `SameSite=Lax` or `Strict` on the session cookie: the cross-site iframe request would not carry it, so the framed portal is logged out. Not complete: the attack could target an action that does not need a session (e.g. submitting a form that sets a password) and the framing itself still leaks UI; use frame-ancestors.

3 Lecture 2b: Web Privacy II, Stateless Tracking and Protections

3.1 Web evolution and the attack surface

Core (what the slides say)

Browsers went from Netscape Communicator 5.0 (1998, first open-source browser, ≈742k lines of code) to Chromium 120 (January 2024, ≈31M lines). APIs keep growing (GPU, gyroscope, accelerometer, gamepad, USB, Bluetooth, AR/VR; generative AI assistants like Copilot, Brave Leo, Arc Max). A 1995 browser exposed browser name and language; a 2024 browser exposes OS, screen, language, time zone, graphics card, audio, video, 3D rendering, real-time communication, payments… "What about security and privacy?"

Go deeper: details she can ask about

- The argument is causal: more APIs ⇒ more *properties exposed* ⇒ more entropy in the fingerprint. Each hardware API is both a feature and a fingerprinting surface (the Battery API is the worked example).
- Numbers she may ask: 742,000 vs 31,000,000 lines (≈ 40×), 1998 vs 2024.

3.2 Browser fingerprinting: definition, stateful vs stateless

Core (what the slides say)

A **browser fingerprint** is a set of information related to a user's device, from the hardware to the OS to the browser and its configuration; **browser fingerprinting** is collecting that information through a browser to build a fingerprint of the device. **Stateful** tracking: the site stores or assigns state, then reads it again later (cookies, local storage, advertising IDs): "I recognise you because I gave/stored an identifier before." **Stateless** tracking: the site recomputes a recognisable pattern from properties exposed *now*: "I recognise you from characteristics you reveal each time." Clearing cookies removes state but does not stop a fingerprint from being recomputed. Example attributes: User-Agent (`Mozilla/5.0 (Windows NT 10.0; Win64; x64) ...Chrome/121`), HTTP headers (Accept, Accept-Encoding, Accept-Language), fonts, platform (`Win32`), screen resolution (`3840x2160x24`), timezone (−480, UTC+8), hardware concurrency (4), battery level (38%), WebGL vendor/renderer (NVIDIA, GeForce GTX 4070 Ti), canvas, browser extensions.

Go deeper: details she can ask about

- The decisive test: *where does the identifier come from?* Stored by the site (stateful) or derived from the browser's current properties (stateless). An ETag is stateful (stored in the cache) even though it is not a cookie; a canvas hash is stateless even though it looks like an ID.
- TP distinguishes three collection modes: *passive* (headers and IP the server receives without running code: User-Agent, Accept-Language, Accept-Encoding, client hints), *active* (JavaScript queries APIs: `navigator`, `screen`, `window`, `Intl`, `document.referrer`), *behavioural* (how the user acts: typing speed, key-hold time, inter-key interval, pauses, corrections). Behavioural signals are probabilistic (keyboard, fatigue, stress change them).
- A fingerprint hash (SHA-256 of features in a fixed order) gives the same ID for the same combination, is short to compare, and hides the raw features from the user (`/collect?fp=8b91c7a4...`); this is why it is hard for ordinary users to notice.
- Fingerprints identify a *browser instance/device*, not a named person (Quiz 1 Q11 A is false).

Hard questions, answered

Q. Quiz 1, Q13. Before any JavaScript runs, the server receives the IP, User-Agent, Accept-Language, Accept-Encoding and client hints.
A. (A) The server can observe these directly from the HTTP request: true. (B) Screen resolution and fonts are always in request headers: false (active collection needed). (C) Standard headers cannot help distinguish browsers: false. (D) A cookie must be stored first: false. Answer: A.

Q. Quiz 1, Q11. Test 4 (everything cleared) yields new IDs 5678 but fingerprint 88 and profile 3; Test 5 (other device) gives 9012, fingerprint 42, profile 9.
A. (A) Fingerprint 88 identifies Alice as a named person: false. (B) Test 4 is consistent with stateless relinking by fingerprint 88: true. (C) New values prove nothing was cleared: false (new IDs show the stores *were* cleared). (D) Different browsers could share fingerprint 88, so linking may give false positives: true. Answer: B and D.

Q. Alice disables JavaScript. Which fingerprinting survives?
A. Passive: IP address and HTTP headers (User-Agent, Accept-*, client hints) are sent on every request; and stateful mechanisms that need no script (cookies, ETag). Active (canvas, WebGL, screen) and behavioural collection stop.

Q. Why is a hashed fingerprint not a "permanent" identifier?
A. Any change in one feature (browser update changes the User-Agent, a new font, a resized window if included) changes the hash; linking over time therefore needs *linkability* algorithms (FP-Stalker) rather than exact matching.

3.3 Specific attributes: Battery, User-Agent, canvas, GPU, mobile, IP

Core (what the slides say)

Battery Status API (first browser 2012): exposes charge level, charging state, estimated time; Olejnik, Acar, Castelluccia & Diaz, *The Leaking Battery* (2015): a third-party tracker on site A sees "73%, 14,237 s remaining, not charging" and 20 seconds later on site B "73%, 14,210 s": the combination changes predictably over a short interval, so it is a *short-term* identifier linking two visits (no cookie needed); it could also be abused to raise prices (low battery). **User-Agent**: historically grew from `NCSA_Mosaic/2.0 (Windows 3.1)` and `Mozilla/1.22 (compatible; MSIE 2.0; Windows 95)` into a long string. **Screen resolution**. **Canvas fingerprinting** (Mowery & Shacham, W2SP 2012): JavaScript asks the browser to draw hidden text and shapes; final pixels vary slightly with browser, OS, fonts, GPU and graphics stack; read the pixels, hash them, use as an attribute (draw → render → read → hash). **GPU fingerprinting**, DrawnApart (Laor et al., NDSS 2022): can two machines with the same model/OS/browser/GPU be told apart? Physical chips are never identical; tiny manufacturing variations make some execution units slightly faster; JavaScript measures the timing trace; a deep-learning model compresses traces into an embedding and compares (same device: similar pattern despite noise). Result on 2,550 devices, 370,392 fingerprints over 7 months (crowd-sourced, "in the wild"): median tracking duration in the 7-day revisit setting rises from 17.5 days (FP-Stalker) to 28 days with DrawnApart (+10.5 days, ≈ 60% longer). Mobile fingerprints are also unique (S&P'16), depending on vendor/model, with different attributes (User-Agent, emojis, canvas). `amiunique.org` (launched November 2014) shows your fingerprint and its evolution via an extension. **IPv4**: ≈4 billion addresses, distributed by region; DHCP leases are negotiated and released. WWW'20 study: 87% of devices kept the same IPv4 for more than 30 days (5,443 devices / 41,566 IP addresses), country/ISP dependent, highly discriminating (93% unique); address cycles (home–work–home), 20% with 30-day-plus cycles.

Go deeper: details she can ask about

- Battery is the model answer for "a signal that is not an identifier but still links visits": stable for seconds to minutes, predictable drift, no storage. Key phrase: *temporary signal, short-term linking.*
- Canvas: the *drawing instructions are identical*; the *rendering* differs (anti-aliasing, font hinting, GPU). Quiz 1 Q14: it does not store an ID in the canvas, does not retrieve viewed images, and does not read a hardware serial number; it renders predefined content, reads pixels and hashes them (answer B).
- DrawnApart is the only *hardware-level* attribute on the slides: it separates devices that are identical in software. The claimed improvement is in *linkability duration*, not uniqueness.
- Why the IP is a poor-but-useful attribute: stable for most devices for weeks, but shared (NAT, campus) and changing with location; the slide's "93% unique" is for the device-to-IP graph in that study, not for a single IP being unique to one person.

Hard questions, answered

Q. Explain how the Battery API links two visits without any identifier, and why the link expires.
A. The tracker on site A records (level, time remaining, charging). Twenty seconds later on site B the same device shows the same level and a time-remaining value lower by about twenty seconds; few devices match that combination at that instant, so the two visits are probably the same device. After minutes the battery drifts and the combination is no longer distinctive; it links visits close in time only.

Q. Two laptops, same model, same OS, same browser version, same GPU model. Which slide attribute can still separate them, and how?
A. DrawnApart: timing traces of GPU execution units reveal manufacturing variations; a learned embedding of the trace is stable per device and different across devices.

Q. Why does canvas fingerprinting need JavaScript while the User-Agent does not?
A. Canvas requires executing drawing code in the browser and reading back pixels (active); the User-Agent is a header the browser attaches to every request (passive).

3.4 Properties: uniqueness, linkability, consistency; entropy

Core (what the slides say)

Three research properties: **uniqueness** (how many fingerprints are unique), **linkability** (can fingerprints of the same device be linked over time as they evolve), **consistency** (do attributes agree with each other; what does an inconsistency reveal). Uniqueness studies: 470,161 fingerprints, 94.2% unique (Panopticlick); 118,934 fingerprints, 89.4% unique (*Beauty and the Beast*, S&P'16): tracking is possible; *Hiding in the Crowd* (WWW'18): 1,816,776 desktop fingerprints, only 35.7% unique. **Shannon entropy** measures how much identifying information a fingerprint (or attribute) contains: $H(X) = -\sum_i P(x_i) \log_2 P(x_i)$; low entropy = many users share the value, high entropy = rarer, identifies more precisely. Linkability: FP-Stalker (S&P'18) tracks users on average over 55 days with the right linking algorithm; DrawnApart (NDSS'22) extends it; Blink (SEAMS'15) generates a random disposable session (OS, browser, fonts) each time to break linkability; FPRandom (ESSoS 2017) adds controlled noise to canvas (implemented in Brave). Consistency: FP-Scanner (USENIX Security'18): most fingerprint-modifying tools are detectable and some leave unique traces per user. Research map: uniqueness (Hiding in the Crowd, Beauty and the Beast, Fingerprinting in Style USENIX Sec'21), linkability (FP-Stalker, DrawnApart, Blink, FPRandom), consistency (FP-Scanner, Taming the Shape Shifter DIMVA'20, Web Runner 2049 DIMVA'20, Morellian analysis DIMVA'19, FP-Redemption DIMVA'21).

Worked numbers: entropy of the OS attribute

100 fingerprints: 50 Windows, 25 Linux, 25 macOS. $P = (0.5, 0.25, 0.25)$.

$$H = -(0.5 \log_2 0.5 + 0.25 \log_2 0.25 + 0.25 \log_2 0.25) = 0.5 \cdot 1 + 0.25 \cdot 2 + 0.25 \cdot 2 = 1.5 \text{ bits.}$$

Quiz 1 Q15: answer C. Checks: a uniform attribute over n values has $\log_2 n$ bits (4 equally likely OSes: 2 bits); an attribute everyone shares has 0 bits; the maximum for 3 values is $\log_2 3 \approx 1.585$ bits, so 1.5 is close to maximal. Entropies of independent attributes add; $\log_2(7 \times 10^9) \approx 33$ bits suffice to single out one device on Earth.

Go deeper: details she can ask about

- Why uniqueness fell from 94.2% to 35.7%: much larger and more diverse sample (nearly 2 million desktop fingerprints collected on a popular site), fewer high-entropy attributes (Flash and Java plugins disappeared, fonts enumerated less), and homogeneous populations (same OS/browser version). The conclusion is nuanced: tracking by exact fingerprint is harder on desktop, but linkability over time still works.
- *Linkability* is about *evolution*: fingerprints change (updates); FP-Stalker's number (55 days average) is the length of time a browser can be followed through changes.
- *Consistency* cuts both ways: spoofing the User-Agent to say Windows while fonts, platform and WebGL say Linux creates a detectable *inconsistency*; the defence itself becomes a fingerprinting signal ("poorly designed spoofing is counterproductive").
- Entropy pitfalls: the quiz asks for the entropy of *one attribute*; use $\log_2$, not ln; do not forget the minus sign; probabilities must sum to 1.

Hard questions, answered

Q. Compute the entropy of a time-zone attribute where 80% are UTC+1, 10% UTC+0, 10% UTC+2, and compare with the OS example.
A. $H = -(0.8 \log_2 0.8 + 2 \times 0.1 \log_2 0.1) = 0.8 \times 0.322 + 0.2 \times 3.322 = 0.258 + 0.664 = 0.92$ bits: less identifying than the OS attribute (1.5 bits) even though it also has three values, because the distribution is skewed.

Q. A defence randomises the canvas output on every page load. Which property does it attack, and what can go wrong?
A. Linkability (and uniqueness of the canvas attribute). Risk: if the noise is naive, FP-Scanner-style consistency checks detect the tool, and a tool used by few people becomes itself a rare, identifying attribute; FPRandom's "controlled" noise and Brave's site-dependent randomisation are designed to avoid that.

Q. Why is 35.7% unique still a privacy problem?
A. The remaining fingerprints are shared among small groups; combined with IP, login state or short-term signals (battery), the group collapses to one device. And linkability (following a changing fingerprint for weeks) does not require global uniqueness.

3.5 Fingerprinting for security

Core (what the slides say)

Tracking asks "who are you across the Web?" (no account needed, the fingerprint acts as an identifier: privacy risk). Security asks "does this login look familiar?" (the account is known; compare the current browser with past logins; the fingerprint is a *risk signal*, not proof of identity: defensive signal).

Hard questions, answered

Q. Why must a fingerprint be a "risk signal, not proof of identity" in authentication?
A. Fingerprints are not unique (false positives) and change over time (false negatives); a mismatch should trigger a step-up (MFA), never a silent denial or a silent acceptance.

3.6 Protection: filter lists and behaviour-based blocking

Core (what the slides say)

A **filter list** is a maintained set of rules describing network requests or page elements a blocker should block, allow, redirect or hide: Disconnect list (used by Firefox, Brave), EasyList (ads) and EasyPrivacy (tracking) (used by AdBlock Plus, uBlock Origin). EasyList syntax: a normal rule blocks every request matching its pattern: `||tracker.com^` (`||` = the domain starts here; `^` = the domain ends here or is followed by a separator such as `/`); a rule beginning with `@@` is an *exception* (allow something a broad rule would break). Limitations: *coverage* (an unlisted tracker loads), *evasion* (rotating domains, changing URLs, first-party/CNAME techniques), *breakage* (aggressive blocking stops login, video, payment, embeds), *maintenance* (continuous measurement, reports, updates). Filter lists are *knowledge-based* ("we already know this domain is a tracker"); **behaviour-based** protection asks "what is this third party actually doing: appearing across many sites, trying to recognise the same browser, storing identifying information?" **Privacy Badger** (EFF): browser extension focused on third-party tracking; uses rules describing tracking behaviour (identifying cookies, local-storage "supercookies", fingerprinting); if the same third party is observed tracking across several unrelated websites it is restricted or blocked. FP-detect: detects which filter lists or blocking extensions are active in a browser (the blocker itself is fingerprintable).

Go deeper: details she can ask about

- EasyList exercise: with `||amazon-adsystem.com^` both `/ad.js` and `/widgets/q?item=10` on `amazon-adsystem.com` match the domain rule and are blocked unless an `@@` exception (e.g. for `/widgets/`) exists; the point of the exercise is that exceptions exist precisely to avoid breakage and that you must check the list, not guess.
- "`||`" anchors at a domain boundary, so `||tracker.com^` also matches `sub.tracker.com` but not `nottracker.com`.
- Privacy Badger's threshold ("several unrelated websites") is a design choice balancing false positives (a CDN seen everywhere but not tracking) against coverage; it needs *identifying* behaviour (cookies with ID-like values), not mere presence.
- Knowledge-based vs behaviour-based is the key contrast; a hybrid is what browsers ship.

Hard questions, answered

Q. A tracker is served from `static.publisher.com`, a first-party subdomain. Which filter-list limitation is this, and which protection can still catch it?
A. Evasion via first-party/CNAME techniques. DNS-level inspection (Firefox resolves the CNAME and sees `bad.tracker.com`) or behaviour-based detection if the subdomain exhibits tracking behaviour across sites (it usually appears on one site only, so even Privacy Badger may miss it).

Q. Why can installing a blocking extension make you *more* fingerprintable?
A. Blockers change observable behaviour (which requests fail, which elements are hidden); FP-detect infers which lists/extensions are active, adding a rare attribute to the fingerprint.

Q. Write a rule blocking everything from `ads.example.net` except its `/consent/` path.

A. `||ads.example.net^` and `@@||ads.example.net/consent/`.

3.7 Browsers against tracking: third-party cookies, Privacy Sandbox, isolation, randomisation

Core (what the slides say)

Google announced (January 2020, Chromium blog) the end of third-party cookies: planned Q2 2022, delayed to Q4 2023, then Q4 2024, "the biggest change in ad tech since its inception". **Privacy Sandbox** client-side APIs: interest-based targeting = **Topics API**; retargeting = Protected Audiences API; frequency capping = Shared Storage API; attribution = Attribution Reporting API. Topics: each week the browser derives 5 topics of interest from browsing history and adds a random one; 469 possible topics (taxonomy v2). Critics: Apple and Mozilla; PETS'24: advertisers and publishers can abuse Topics to assign unique identifiers; "privacy objectives and design are directly at odds with the natural diversity of user behaviours." Alternatives: advertisers invested in their own user IDs; device data; first-party data (precise, easy, compatible with legislation); **Conversions APIs** (Meta, Amazon, Microsoft Offline Conversions, Google Enhanced Conversions, TikTok Events, X). Status: 2020 phase-out announced; 2024 Chrome moved to a user-choice approach; 2025–2026 Google confirmed no universal phase-out (latest official direction). Lesson: browser privacy is not only technical; compatibility, advertising economics, standards and deployment shape what reaches users. **Firefox Total Cookie Protection**: each top-level site gets its own cookie jar; a third party embedded on site A cannot reuse the same jar on site B (same tracker ≠ same cross-site cookie context); targets stateful cross-site tracking, fingerprinting needs separate defences. **Brave**: add subtle, site-dependent randomness to selected fingerprintable outputs so repeated measurements are less useful as a stable identifier (citing PriVaricator and FPRandom). **FP-Scanner**: changing only some values creates inconsistencies (UA says Windows, fonts/platform/WebGL say Linux); trackers check coherence; inconsistencies reveal the tool and sometimes the real OS/browser; poorly designed spoofing is counterproductive. **CNAME cloaking**: `f0Ab1ze.mywebsite.com CNAME bad.tracker.com`, `bad.tracker.com A 119.183.2.123`; the tracker looks first party, bypasses ad blockers in most browsers (Firefox includes DNS-level APIs); a threat to transparency because everything is hidden server-side; regulators (CNIL) must audit companies.

Go deeper: details she can ask about

- Topics mechanics: 5 topics + 1 random per week, 469 topics; the random topic is plausible deniability; the PETS'24 abuse is that the *sequence* of weekly topic sets observed across sites can be combined into a quasi-identifier, which is a stateless-tracking argument applied to a privacy API.
- Timeline dates are examinable: 2020 announce; 2022 Q2 planned; 2023 Q4; 2024 Q4; 2024 user choice; 2025/2026 no phase-out.
- Total Cookie Protection partitions *by top-level site*; the tracker still gets a cookie on each site, but a *different* one, so it degrades to first-party-analytics-like visibility. It does nothing against fingerprinting, CNAME-cloaked first-party cookies, or cookie syncing via login.
- Brave's randomisation is *site-dependent* ("farbling"): the same site sees a stable value (so the site works) but different sites see different values (so they cannot link). That is the answer to "why not just randomise everything?"
- CNAME cloaking needs DNS to detect: the browser sees a first-party hostname; only resolving the CNAME chain reveals the tracker; filter lists operating on URLs miss it unless they know the specific subdomain.

Hard questions, answered

Q. Quiz 1, Q6. `news.alpha.test` and `shop.beta.test` embed a pixel from `tracker.test` with `tid=T9; SameSite=None; Secure`; third-party cookies allowed.
A. (A) Requests to `tracker.test` from both publishers can include the same `tid`: true. (B) A publisher's JavaScript can read `tid` via `document.cookie`: false (different origin). (C) The same-origin policy prevents sending `tid`: false (SOP governs script access, not cookie attachment). (D) The tracker can associate both visits with one profile: true. Answer: A and D. With Total Cookie Protection the answer to (A) and (D) flips: two jars, two different `tid` values.

Q. Why does Firefox's cookie partitioning stop cross-site tracking but not cookie syncing between two trackers embedded on the *same* site?
A. Partitioning is per top-level site; on one site both trackers have cookies in that site's jar, the redirect sync still links them within that site. It stops the *reuse* of one tracker's cookie across sites.

Q. Compare the two defence philosophies: Firefox (isolate state) vs Brave (randomise outputs). Which threat does each address?
A. Firefox: stateful cross-site tracking (cookies/storage reuse). Brave: stateless tracking (fingerprint stability). A complete browser needs both, plus list/behaviour blocking for known trackers.

Q. Why is CNAME cloaking "a big threat for transparency"?
A. The mapping from the first-party-looking hostname to the tracker lives in DNS and server configuration; nothing in the page reveals it; users and even blockers see a first-party request; only DNS inspection or regulator audits can expose it.

Q. Topics API: a site observes your weekly topic sets for 10 weeks. Why can this approximate an identifier?
A. Each week yields a set drawn from 469 topics; the sequence of sets over weeks is highly specific to one browsing history; two sites observing the same sequence can link the browser, which is the PETS'24 critique.

4 Lecture 3: Introduction to Cryptography, Classical and Symmetric

4.1 Terminology and goals

Core (what the slides say)

Cryptography is everywhere: HTTPS, end-to-end messaging, cloud and full-disk encryption (BitLocker), password hashing, network authentication, certificates and signatures. **Cryptology** (kryptós + lógos, "the study of the hidden") = **cryptography** (secret writing to hide meaning, confidentiality) + **cryptanalysis** (breaking cryptosystems). Cryptography: techniques for secure communication in the presence of adversaries (Eve, "a very powerful, smart person"). Used for **confidentiality** (protection against eavesdroppers), **data integrity** (content not altered, deliberately or accidentally), **data authentication** (the message is from the sender it claims). Plaintext, ciphertext, cipher (algorithm both ways), key (word/phrase/bits that modifies enciphering/deciphering). **Kerckhoffs's principle**: a cryptographic system should remain secure even if everything about it, except the key, is public. Cryptosystem: $c = E(m, k_e)$, $m = D(c, k_d)$; message space M, ciphertext space C, key space K. Modern ciphers: public algorithm + secret key; in the past simple algorithms with long keys, today sophisticated algorithms with shorter keys. Classification: symmetric ($k_e = k_d$; everyone who knows k knows the full secret), asymmetric ($k_e \neq k_d$), hash functions. Symmetric ciphers: *stream* (one bit/byte at a time, continuous

data) and *block* (fixed-size blocks, same key, output block same size). Ciphers encrypt for *confidentiality*.

Go deeper: details she can ask about

- Kerckhoffs is the justification for public standards (DES, AES competitions) and the reason "secret algorithm" is a red flag: the algorithm leaks (reverse engineering), the key can be replaced.
- Confidentiality, integrity and authentication are *separate*; encryption alone gives only the first (Lecture 5 insists: "confidentiality and authenticity are separate things").
- The key-exchange problem motivates Lecture 5: symmetric crypto needs a *secure channel* to share k (the diagram's second channel).

Hard questions, answered

Q. State Kerckhoffs's principle and give two practical consequences.
A. Security must rest only on the secrecy of the key. Consequences: algorithms can be published and reviewed (AES selected by open competition); a compromised key is replaced without redesigning the system; "security by obscurity" is not a design.

Q. Does encryption provide integrity? Give the one-line counterexample from Lecture 5.
A. No. Eve replaces $(m, H(m))$ by $(m', H(m'))$ and Bob's check passes; for encryption, flipping ciphertext bits (e.g. in XOR/stream modes) flips plaintext bits undetected. Integrity needs a MAC or signature.

4.2 Diffie–Hellman and the Merkle story

Core (what the slides say)

1974: a Stanford student (Ralph Merkle) proposes public-key cryptography; the professor shoots it down; 1975 his paper is rejected by Communications of the ACM ("not in the main stream of present cryptography thinking. . . extremely dangerous to transmit key information in the clear"); today he is "a father of cryptography". **Diffie–Hellman key exchange** (figure): Alice and Bob agree on public g and p, pick secrets a, b, exchange $g^a \bmod p$ and $g^b \bmod p$, both compute $g^{ab} \bmod p$. An eavesdropper would need to solve $6^x \bmod 13 = 2$ or $6^x \bmod 13 = 9$ (the discrete logarithm), "which is hard".

Go deeper: details she can ask about

- The slide example uses $p = 13$, $g = 6$; the hard problem is the *discrete logarithm*, not factoring (that is RSA). With $p = 13$ it is trivially brute-forced; hardness needs large p.
- DH alone is vulnerable to the *man in the middle* (Lecture 5): Eve negotiates k_1 with Alice and k_2 with Bob and relays; "need to authenticate endpoints" (signatures/certificates).

Hard questions, answered

Q. In the slide's example, find x with $6^x \bmod 13 = 2$ by hand and say why this does not contradict "hard".
A. $6^1 = 6$, $6^2 = 36 \equiv 10$, $6^3 \equiv 60 \equiv 8$, $6^4 \equiv 48 \equiv 9$, $6^5 \equiv 54 \equiv 2$: $x = 5$ (and $x = 4$ gives 9). Brute force costs up to $p - 1$ steps; with a 2048-bit p that is astronomically many; hardness is asymptotic.

Q. What does DH give and what does it not give?

A. A shared secret over a public channel (confidentiality of the key). Not authentication: without signing the exchanged values, a MITM is undetectable.

4.3 Classical ciphers and how to break them

Core (what the slides say)

Caesar: shift the alphabet (fixed $K = 3$: `Security is very interesting` → `VHFXULWB LU YHRB LQWHUHVWLQJ`; `JGNNQ YQTNF` decrypts with $K = 2$ to `HELLO WORLD`). **Brute force**: try keys until the plaintext is meaningful; **key space**: all possible keys (Caesar: 26 shifts; try $0 \ldots 25$). **Simple substitution**: each letter replaced by a fixed different letter (`send reinforcements` → `ktdpjtfdoejytbtdlk`); key space $26! \approx 4.03 \times 10^{26}$, brute force impractical, yet insecure: **letter frequency** (most common e, t, a, o, i, n; least j, x, q, z); `GSRH RH Z HVXIVG` is the Atbash-like reversal = `THIS IS A SECRET`. **Vigenère**: a set of Caesar ciphers chosen by a keyword (`send reinforcements` with key `somesecret` → `kszhjikejhjqqqwrvj`). Attacks on cryptography: *analytical* (cleverness: weaknesses in the algorithm or its use), *brute force* (money: try all keys, parallelisable; if cost to crack exceeds the value, the system is *computationally secure*), *social engineering* (the person), *implementation attacks* (software, hardware, usage). Enigma: German naval advantage in WWII, broken by Turing's *bombe*. Strengthening: with computers, substitution and transposition alone became obsolete; combine substitution then several transpositions so ciphertext looks random and statistical relations vanish (DES, AES).

Go deeper: details she can ask about

- Key-space size is necessary, not sufficient: substitution has 26! keys and falls to frequency analysis in minutes. The exam question "is a large key space enough?" wants exactly this.
- Vigenère attack recipe (TP): guess/know the key length L; split the ciphertext into L groups (`text[i::L]`); each group is a Caesar cipher with one key letter; in each group the most frequent letter is probably `E`; shift = (most frequent − E) mod 26; key letter = that shift (E→G means shift 2, key `C`). Finding L when unknown: repeated trigrams at distances that are multiples of L (Kasiski) or index of coincidence.
- Frequency analysis gives *hypotheses*: combine with short words (`A`, `THE`), doubled letters, repeated patterns, and context (TP hint order E > T > R > S/O > A > H > N > D > I > G > L for that specific text).
- "Computationally secure" is an economic definition (cost > value), distinct from "unconditionally secure" (one-time pad), which the course does not treat but may mention.

Worked numbers: Caesar and substitution from the TP

Caesar $K = 4$: `CRYPTO IS FUN UNTIL THE PROFESSOR SAYS QUIZ` → `GVCTXS MW JYR YRXMP XLI TVSJIWWSV WECW UYMD`. Caesar $K = 8$ ciphertext `GWCL JMBBMZ VWB JM CAQVO BWWTA BW AWTDM BPQA` → `YOUD BETTER NOT BE USING TOOLS TO SOLVE THIS`. Unknown key: `ESP VPJ TD FYVYZHY ECJ MCFEP QZCNP` is shift 11 → `THE KEY IS UNKNOWN TRY BRUTE FORCE`. Substitution table (A→Q, B→M, C→J, D→Z, E→T, ...): `KEEP THE SECRET SAFE` → `LTTN EKT YTJRTE YQGT`; `ERVYE DXVR SXFPJ` → `TRUST YOUR LOGIC`.

Hard questions, answered

Q. Why can a substitution cipher with 26! keys be broken faster than Caesar with 26 keys is broken by brute force?

A. Because the attack is analytical, not exhaustive: language statistics survive substitution (each plaintext letter maps to one ciphertext letter, so frequencies are permuted, not destroyed). Caesar brute force is 26 trials; substitution frequency analysis is a few dozen educated guesses.

Q. A Vigenère key as long as the message and never reused: still breakable by the group method?
A. No: each group has one letter, no statistics; this is the one-time pad. The group attack needs the key to *repeat*.

Q. Classify: a side-channel reading power consumption of a smart card; bribing the operator; trying all 2^{56} DES keys; exploiting ECB's block independence.
A. Implementation attack; social engineering; brute force; analytical attack (weakness in the manner of use).

4.4 XOR and the attack models

Core (what the slides say)

XOR: "different → 1, same → 0"; XOR-ing by the same thing twice returns the data; XOR itself could be an "encryption". Demo: `Hello` ⊕ repeating key `key` gives `00100011 00000000 00010101 00000111 00001010`; XOR with the key again recovers `Hello`. Attack models (what the cryptanalyst has, with the algorithm known): **ciphertext only**; **known plaintext** (plaintext–ciphertext pairs under the same key); **chosen plaintext** (attacker-chosen plaintexts encrypted); **chosen ciphertext** (attacker-chosen ciphertexts decrypted); **chosen text** (both). Attacking XOR: known plaintext: plaintext ⊕ ciphertext = key; chosen plaintext of all zeros: ciphertext = key; chosen ciphertext of all zeros: "plaintext" = key; ciphertext only: the second ciphertext byte is `00000000`, so the second key byte equals the second plaintext byte (`e` = `e`), "etc."

Go deeper: details she can ask about

- XOR is a *one-time pad* if the key is random, as long as the message, and never reused; it is *broken* the moment the key repeats (Vigenère on bytes): the ciphertext-only attack on the demo works because the 3-byte key repeats and because plaintext has structure.
- Under KPA, any repeating-XOR scheme yields the key immediately; the lesson is that modern ciphers must resist *chosen*-plaintext attacks, the strongest realistic model (the attacker can often make the system encrypt what they want).
- The attack-model hierarchy orders attacker power: COA < KPA < CPA ≈ CCA (chosen text = both). A scheme secure under CCA is secure under the weaker ones.

Hard questions, answered

Q. Two messages are XOR-encrypted with the same one-time key. What can Eve compute without the key?
A. $c_1 \oplus c_2 = m_1 \oplus m_2$; with structured plaintext (text) she can often recover both messages (crib dragging). Key reuse destroys the one-time pad.

Q. Give a realistic situation for each attack model.
A. COA: intercepting radio traffic. KPA: encrypted files with known headers (PDF magic bytes). CPA: a web app that encrypts a cookie value the attacker controls. CCA: a server that decrypts submitted tokens and reveals whether padding was valid (padding oracle).

4.5 Block cipher modes of operation

Core (what the slides say)

How to apply a block cipher to data bigger than one block? **ECB** (electronic codebook): "apply the key to each block"; parallel; *but identical blocks give identical ciphertext*, informative to an attacker (the penguin image). **CBC** (cipher block chaining): each plaintext block is XOR-ed with the previous ciphertext block; prevents patterns even in regular data. **IV** (initialisation vector): random, *not secret* (sent in clear), prevents a past-seen plaintext from producing the same ciphertext in the first block. CBC: encryption parallelisable? no; decryption parallelisable? yes; random read access? yes. **CTR** (counter): full parallelisation; a *nonce* is basically like an IV. Modes list: ECB, CBC, CFB, OFB, CTR, CCM (counter with CBC-MAC).

Go deeper: details she can ask about

- CBC equations: $C_i = E_K(P_i \oplus C_{i-1})$, $C_0 = IV$; $P_i = D_K(C_i) \oplus C_{i-1}$. Decryption of block i needs only C_i and C_{i-1}: parallel and random access; encryption needs C_{i-1} first: sequential.
- CTR: $C_i = P_i \oplus E_K(\text{nonce} \,\|\, i)$; a stream cipher built from a block cipher; both directions parallel; never reuse a nonce with the same key (two-time pad).
- ECB's flaw is *deterministic per block*: equal 16-byte plaintext blocks → equal ciphertext blocks, so the *structure* (not the content) leaks: image outlines, repeated records, dictionary attacks on short fields.
- A random IV makes encryption *probabilistic*: the same message encrypted twice gives different ciphertexts; a fixed IV reintroduces an ECB-like leak for the first block.
- CCM = counter mode for confidentiality + CBC-MAC for integrity: the hint that modes can provide *authenticated* encryption; plain CBC/CTR give confidentiality only.

From the TP: OpenSSL, Mission 2

The file `classified.enc`, AES-256-ECB with an OpenSSL password: the traditional salted format starts with the 8-byte signature `Salted__` followed by an 8-byte *salt* (public, not the key), so 16 bytes must be skipped before analysing ciphertext. The image is $432 \times 475 = 205{,}200$ bytes, exactly 12,825 blocks; PKCS#7 padding then adds a full 16-byte block, so the ciphertext is 205,216 bytes and the file 205,232 bytes. Counting equal 16-byte blocks with `collections.Counter` and giving equal blocks the same grey shade reveals the picture's outline (`pattern_ecb.png`): you reveal a *pattern, not the secret*. Decrypt with `openssl enc -d -aes-256-ecb -pbkdf2 -p -in classified.enc -out recovered.raw`; re-encrypt with `-aes-256-cbc`: repeated-block counts drop to ≈ none because each block is XOR-ed with the previous ciphertext before encryption.

Hard questions, answered

Q. Why does ECB leak the image but not the pixel values?
A. Identical 16-pixel runs encrypt to identical blocks, so equal regions look equal; the actual grey values are hidden behind AES. You learn equality relations, not content.

Q. In CBC, one ciphertext bit of block i is corrupted in transit. Which plaintext blocks are affected and how?
A. Block i is garbled entirely (it passes through D_K); block $i+1$ has exactly that bit flipped (it is XOR-ed with C_i); later blocks are intact. This limited propagation is also why CBC without a MAC allows bit-flipping attacks on block $i+1$.

Q. Why is the IV sent in clear not a weakness, and what *would* be a weakness?
A. The IV only needs to be unpredictable/unique so that equal first blocks differ; secrecy adds nothing because the attacker knowing C_0 does not help invert E_K. Reusing an IV (or using a predictable one under CPA) is the weakness.

Q. Rank ECB, CBC, CTR on: encryption parallelism, decryption parallelism, random access, pattern hiding.
A. ECB: yes, yes, yes, no. CBC: no, yes, yes, yes. CTR: yes, yes, yes, yes (needs unique nonce).

4.6 Symmetric ciphers in use; DES

Core (what the slides say)

Table: DES (key 56, block 64, 1975: "now ATMs can exist"); 3DES (112/168, 64, 1995: "the DES key is too small, duct-tape three together"); Twofish, Serpent, Rijndael/AES (128/192/256, block 128, 1998: "3DES is too slow"). 3DES survives in legacy financial systems (ATMs); AES dominates, implemented in CPU hardware (5× or more faster than software versions of others). **DES**: 64-bit blocks, 56-bit key, symmetric, 16 identical rounds with a different subkey each. Structure: initial permutation IP (a fixed table of *positions*; e.g. output bit 1 ← input bit 58; for input `0x0123456789ABCDEF` the first output byte is `11001100`), 16 Feistel rounds, final permutation IP^{-1}. **Feistel**: $L_i = R_{i-1}$, $R_i = L_{i-1} \oplus f(R_{i-1}, K_i)$. The f-function in four steps: (1) *expansion* E: 32 → 48 bits by repeating boundary bits (for R_0 = `0xF0AAF0AA`, $E(R_0)$ = `011110 100001 010101 010101 011110 100001 010101 010101`); (2) XOR with the 48-bit round key K_1 (gives `0x6117BA866527`); (3) eight *S-boxes* $S_1..S_8$, each 6 bits → 4 bits: *row* from the outer bits b_1b_6, *column* from the middle four $b_2b_3b_4b_5$ (`011000`: row `00` = 0, column `1100` = 12, $S_1[0, 12] = 5 =$ `0101`); output 32 bits `0x5C82B597`; (4) *permutation* P (fixed 8×4 table; rearranges, does not change values; spreads one S-box's bits to different S-boxes next round: diffusion): P(0x5C82B597) = `0x234AA9BB`. Toy round: $L_0 =$ `1010`, $R_0 =$ `1001`, $E' = [4, 1, 2, 3, 4, 1]$, $K_1 =$ `101011`, $P' = [2, 4, 3, 1]$: $E'(R_0) =$ `110011`, $\oplus K_1 =$ `011000`, $S_1 \to$ `0101`, $P' \to$ `1100`, $L_1 = R_0 =$ `1001`, $R_1 = L_0 \oplus$ `1100` = `0110`. **Key schedule**: 64-bit input key = 56 key bits + 8 parity bits (bits 8, 16, ..., 64 unused), removed by permuted choice PC-1; split into 28-bit halves C_0, D_0; rotate each left by 1 in rounds 1, 2, 9, 16 and by 2 in all other rounds; PC-2 selects 48 bits as K_i (each K_i is a permutation of a subset of k). Toy: key `10101101`, PC-1 = $[1, 2, 3, 5, 6, 7] \to$ `101110`, $C_0 =$ `101`, $D_0 =$ `110`; round 1 shift 1: `011`, `101`; PC-2 = $[1, 2, 4, 5]$ on `011101` → $K_1 =$ `0110`; round 2 continues from C_1, D_1 (do not restart): `110`, `011` → `110011` → $K_2 =$ `1101`. **Decryption**: Feistel ciphers only need the key schedule reversed: $K_{16} \dots K_1$; since $C_{16} = C_0$ and $D_{16} = D_0$ (total rotation 28), the reversed schedule uses no rotation in round 1, one bit to the right in rounds 2, 9, 16, two bits right elsewhere; no new keys are generated. Cryptanalysis: $2^{56} \approx 7 \times 10^{16}$ keys; at 10^9 keys/s about 2.3 years on one machine; in 1998 the EFF built a $250,000 machine that cracked DES in under 2 days. 3DES applies DES three times with different keys; obsolete, replaced by AES.

Go deeper: details she can ask about

- Feistel invertibility is *structural*: $L_{i-1} = R_i \oplus f(R_{i-1}, K_i)$ and $R_{i-1} = L_i$; f need not be invertible (and the S-boxes are not). That is why decryption = encryption with reversed keys, and why AES (not Feistel) needs explicit inverse operations.
- IP and P tables list *positions*; they never change bit values; the only non-linear step is the S-box, "crucial for DES security". Expansion duplicates 16 boundary bits (each 4-bit group borrows one neighbour on each side).
- S-box indexing trap: row = b_1b_6 (outer), column = $b_2..b_5$ (inner); the slide's mnemonic: "outside bits = row, inside bits = column".
- Rotation schedule: 1 bit in rounds 1, 2, 9, 16; 2 bits otherwise; $4 \times 1 + 12 \times 2 = 28$ = a full rotation of a 28-bit half, hence $C_{16} = C_0$.
- 3DES keying: 2-key (112 bits effective: K_1, K_2, K_1) or 3-key (168); EDE (encrypt-decrypt-encrypt) so that $K_1 = K_2 = K_3$ reduces to single DES for compatibility. Meet-in-the-middle

makes 2DES pointless ($\approx 2^{57}$), which is why "three".
- 56-bit arithmetic: $2^{56} = 7.2 \times 10^{16}$; $7.2 \times 10^{16} / 10^9 = 7.2 \times 10^7$ s ≈ 2.3 years; the EFF machine did $\approx 9 \times 10^{10}$ keys/s.

Hard questions, answered

Q. Why does DES have a 56-bit key when the input key is 64 bits?
A. Eight bits (positions 8, 16, ..., 64) are parity bits; PC-1 drops them. Effective key space is 2^{56}.

Q. Compute S_1 for input `100110`.
A. Row = $b_1 b_6$ = `10` = 2; column = $b_2 b_3 b_4 b_5$ = `0011` = 3; $S_1[2,3]$ = 8 = `1000` (row 2 of the slide's table: 4 1 14 8 ...).

Q. Explain in one sentence why reversing the subkeys decrypts a Feistel cipher, and why the same trick fails for AES.
A. Each Feistel round is its own inverse up to swapping halves given the same subkey ($L_{i-1} = R_i \oplus f(R_{i-1}, K_i)$), so running rounds backward with $K_{16} \dots K_1$ undoes encryption; AES rounds apply non-involutory operations (SubBytes, MixColumns) that require explicit inverses.

Q. Why is 2DES not used and 3DES's effective strength 112 bits with three applications?
A. Meet-in-the-middle: encrypt a known plaintext under all 2^{56} K_1, decrypt its ciphertext under all 2^{56} K_2, match: cost $\approx 2^{57}$, not 2^{112}. Three applications push the attack to $\approx 2^{112}$.

Q. Was DES "broken" by cryptanalysis or by brute force?
A. Primarily by brute force on a too-short key (EFF 1998); analytical attacks (differential, linear) exist but need enormous known/chosen plaintext; the lesson on the slide is key size.

4.7 AES

Core (what the slides say)

Public competition to replace DES (56-bit keys and 64-bit blocks no longer adequate); Rijndael (Rijmen and Daemen, "Rhine-doll") became AES in 2001 (FIPS-197): 128-bit block (the slide notes an error in Harris p. 809), key sizes 128/192/256, *not* a Feistel network. Units: byte, word (4 bytes), state (16 bytes as a 4×4 matrix, byte b_i at row i mod 4, column $\lfloor i/4 \rfloor$: b_{i+4j}). Rounds depend on key size: 10 (128), 12 (192), 14 (256). Round = **SubBytes** (each byte replaced via the S-box; invertible with the inverse S-box; e.g. `19`→`D4`, `A0`→`E0`, `3D`→`27`, `E3`→`11`, `BE`→`AE`), **ShiftRows** (row r cyclically shifted left by r: row 1 `27 BF B4 41` → `BF B4 41 27`; row 2 `11 98 5D 52` → `5D 52 11 98`; row 3 `AE F1 E5 30` → `30 AE F1 E5`), **MixColumns** (column by column, multiply by the fixed matrix $\begin{smallmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{smallmatrix}$ in GF(2^8): first byte = $(02 \cdot \text{D4}) \oplus (03 \cdot \text{BF}) \oplus 5\text{D} \oplus 30 = 04$; the whole column `D4 BF 5D 30` → `04 66 81 E5`), **AddRoundKey** (XOR with the round key: `04`⊕`A0`=`A4`). Multiplication by 02: shift left one bit; if the first bit was 1, XOR with `1B` (`D4` = `11010100` → `10101000` = `A8`, overflow, ⊕`1B` = `B3`); by 03: $(02 \cdot x) \oplus x$. Key expansion produces the round keys (11 for AES-128).

Worked numbers: MixColumns first byte, as in the TP

$02 \cdot \text{D4} = \text{B3}$ (above). $02 \cdot \text{BF}$: `BF` = `10111111` → `01111110` = `7E`, overflow, ⊕ `1B` = `65`; $03 \cdot \text{BF} = 65 \oplus \text{BF} = \text{DA}$. Then $\text{B3} \oplus \text{DA} = 69$; $69 \oplus 5\text{D} = 34$; $34 \oplus 30 = 04$. ✓

Go deeper: details she can ask about

- Round structure: an initial AddRoundKey, then $N_r - 1$ full rounds, then a final round *without MixColumns*; $N_r + 1$ round keys (11/13/15). Decryption order (TP): AddRoundKey(K_{10}); for rounds 9…1: InvShiftRows, InvSubBytes, AddRoundKey, InvMixColumns; finally InvShiftRows, InvSubBytes, AddRoundKey(K_0).
- *Confusion* (Shannon): make the plaintext–key–ciphertext relation complex; SubBytes is the only non-linear step. *Diffusion*: spread each input byte's influence; ShiftRows moves bytes *between columns*, MixColumns mixes *within a column*; together, after two rounds every byte depends on every input byte. Caesar's $C = (P + k) \bmod 26$ has neither.
- GF(2^8) arithmetic: addition is XOR; multiplication is polynomial multiplication modulo $x^8 + x^4 + x^3 + x + 1$ (`0x11B`), which is why the reduction constant is `1B`; `xtime` is multiplication by x (02). Inverse MixColumns uses the matrix with `0E 0B 0D 09`.
- Key expansion (AES-128): 44 words; $w_0..w_3$ = key; for i divisible by 4: RotWord, SubWord, XOR Rcon[$i/4$]; always XOR with w_{i-4}; Rcon first bytes `01 02 04 08 10 20 40 80 1B 36` (the sequence of 02^k in GF(2^8), which is why `80` is followed by `1B`).
- Why not Feistel: an SPN (substitution–permutation network) processes the *whole* block each round; decryption needs the inverse S-box, right rotations and the inverse matrix (the TP implements all four).
- Table memory: DES 1975 / 3DES 1995 / AES finalists 1998, standard 2001.

From the TP: AES without diffusion (Act I)

A two-round byte cipher $c = S(S(p \oplus k_0) \oplus k_1) \oplus k_2$ with ShiftRows and MixColumns removed encrypts each byte *independently*. From pair (00, A4): $k_2 = \text{A4} \oplus S(S(00 \oplus k_0) \oplus k_1)$, a function of (k_0, k_1). Enumerate the 2^{16} pairs (k_0, k_1), derive k_2, keep those consistent with the other two pairs: a 2^{16} search instead of 2^{24} (and instead of 2^{128} for real AES). Without diffusion, each byte position can be attacked *separately* with a few known plaintexts; diffusion forces the attacker to treat the whole 16-byte state at once. Also: apply AddRoundKey with the all-`0F` key to S: every byte XOR `0F` (`00`→`0F`, `10`→`1F`, `23`→`2C`…); SubBytes on `00 01 02 03` → `63 7C 77 7B`; ShiftRows of the index matrix: row 1 `11 12 13 10`, row 2 `22 23 20 21`, row 3 `33 30 31 32`.

Hard questions, answered

Q. Compute $02 \cdot 57$ and $03 \cdot 57$ in GF(2^8).
A. `57` = `01010111`; shift → `10101110` = `AE`; top bit was 0, no reduction: $02 \cdot 57 = \text{AE}$. $03 \cdot 57 = \text{AE} \oplus 57 = \text{F9}$.

Q. Which AES step provides confusion and which provide diffusion? What breaks if you remove each?
A. SubBytes: confusion (remove it and AES becomes linear, solvable by linear algebra). ShiftRows + MixColumns: diffusion (remove them and each byte is encrypted independently; the TP shows a 2^{16} attack per byte position). AddRoundKey: injects the key (remove it and there is no secret).

Q. Why does the final round omit MixColumns, and does that weaken AES?
A. MixColumns is linear and public; applied last it would be undone trivially by the attacker, adding no security; omitting it makes encryption and decryption structurally symmetric. No weakening.

Q. Alice uses AES-256 in ECB mode for a database of 16-byte customer IDs. Strong cipher, weak system: explain.
A. Equal IDs produce equal ciphertexts, so frequency and equality analysis works on the ciphertext without touching AES (the TP's image lesson: "strong encryption algorithms can still be used insecurely"). Use CBC/CTR with random IV, or authenticated encryption.

Q. AES-128 vs AES-256: what changes and what does not?
A. Key length (128 vs 256), number of rounds (10 vs 14), number of round keys (11 vs 15), key-expansion details. Block size (128) and the four transformations do not change.

5 Lecture 5: Asymmetric Cryptography, Hashing, Authentication, PKI, Steganography

5.1 Why asymmetric: key management and the mailbox idea

Core (what the slides say)

Symmetric analogy: a safe with a strong lock; only Alice and Bob have a copy of the key. Limitations: key management and the need for a secure channel for key exchange; N people need $N(N-1)/2$ keys (6 people: 15 keys); you want to send a message without a pre-shared key. Mailbox principle: everyone can drop a letter, only the owner has the key to open the box. First publication 1976 by Diffie and Hellman, and Merkle. Each user has a public key and a private key; for N users $2N$ keys (6 users: 12). Properties: computationally *easy* to create key pairs, to encrypt knowing the public key, to decrypt knowing the private key; computationally *infeasible* to derive the private key from the public key (or vice versa) or otherwise recover the message. Based on **one-way functions** ($y = f(x)$ easy, x from y hard) with a **trapdoor** (secret information t that makes inversion easy; generated by an algorithm Gen; f computable in probabilistic polynomial time). Mathematical challenge: find a one-way function and a matching trapdoor; candidate: integer factorisation (multiplying two large primes is easy, recovering them from the product is hard). Toy: $6895601 = 1931 \times 3571$; with the trapdoor $t = 1931$ the division is immediate.

Go deeper: details she can ask about

- Key-count arithmetic is a favourite: symmetric $\binom{N}{2} = N(N-1)/2$ grows quadratically; asymmetric $2N$ linearly; for $N = 100$: 4950 vs 200.
- "Computationally infeasible" means no known polynomial-time algorithm, not impossible: the RSA Factoring Challenge table shows the frontier moving (RSA-768 factored in December 2009, RSA-250 (829 bits) in February 2020; 1024-bit keys "are out there"; 2048-bit is the default now). RSA is "cool but slow and uses giant keys".
- Trapdoor vocabulary: the public key is f; the private key is the trapdoor t.

Hard questions, answered

Q. Why does public-key cryptography not eliminate the key-distribution problem but transform it?
A. You no longer need a *secret* channel (public keys are public), but you need an *authentic* one: Alice must be sure the public key really is Bob's, otherwise a MITM substitutes his own. That is the problem certificates solve.

Q. Give the three computational requirements and the two infeasibility requirements of a public-key system.
A. Easy: key generation; encryption with the public key; decryption with the private key. Infeasible: recovering the private key from the public key; recovering the message from the ciphertext and public key.

5.2 RSA

Core (what the slides say)

Rivest, Shamir, Adleman (1977). Public key $\{e, n\}$, private key $\{d, n\}$. Encrypt $c = p^e \bmod n$, decrypt $p = c^d \bmod n$. Key generation: choose distinct primes p, q (secret, random, similar magnitude); $n = pq$ (modulus, part of both keys); $\varphi = (p-1)(q-1)$; choose e with $\gcd(e, \varphi) = 1$, $1 < e < \varphi$ (e need not be secret); find d with $de \equiv 1 \pmod{\varphi}$ (d private). Slide example: primes 3 and 11, $n = 33$, $\varphi = 20$, $e = 3$ ($\gcd(3, 20) = 1$), $d = 7$ ($3 \times 7 = 21 \equiv 1 \bmod 20$); public $(33, 3)$, private $(33, 7)$. Encrypt $p = 4$: $4^3 = 64 \bmod 33 = 31$; decrypt $31^7 \bmod 33 = 4$. Exercise $p = 5$: $5^3 = 125 \bmod 33 = 26$; $26^7 \bmod 33$: $26^2 \equiv 16$, $26^4 \equiv 25$, $26^7 = 25 \times 16 \times 26 \bmod 33 = 5$. Security rests on factoring n.

Worked numbers: TP OpenSSL Mission 1: $p = 5$, $q = 11$, $e = 3$, $C = 23$

$n = 55$, $\varphi = 4 \times 10 = 40$; d: $3d \equiv 1 \pmod{40}$, try $d = 27$ ($81 = 2 \times 40 + 1$). Decrypt: $23^2 = 529 \equiv 529 - 9 \times 55 = 34$; $23^4 \equiv 34^2 = 1156 \equiv 1156 - 21 \times 55 = 1$; so $23^{27} = (23^4)^6 \cdot 23^3 \equiv 23^3 = 23 \times 34 = 782 \equiv 782 - 14 \times 55 = 12$. Password $m = 12$. Verify: $12^3 = 1728 = 31 \times 55 + 23 \equiv 23 = C$. ✓ Weak-RSA part: a 50-bit modulus is factored by trial division up to $\sqrt{n} \approx 2^{25} \approx 3.4 \times 10^7$ candidates (seconds in Python); a 512-bit modulus has $\sqrt{n} \approx 2^{256}$, hopeless by trial division (though 512-bit RSA is factorable with number-field-sieve methods, which is why it is deprecated).

Go deeper: details she can ask about

- The slide says "exponentiation of integers modulo a prime"; precisely, modulo $n = pq$ (a product of two primes). Correctness: $p^{ed} \equiv p \pmod{n}$ because $ed = 1 + k\varphi$ and Euler/Fermat.
- Why e must be coprime to φ: otherwise d does not exist (no inverse modulo φ). Common $e = 65537 = 2^{16} + 1$; the toy $e = 3$ works when $\gcd = 1$.
- Finding d in general uses the extended Euclidean algorithm; for small numbers "try successive d" (TP hint).
- Modular exponentiation by repeated squaring is the only practical method; the slide's optional check (26^2, 26^4, then combine) is exactly that.
- RSA is deterministic and slow: encrypt only short data (a key) and use padding in practice; the TP's "RSA cannot encrypt large files" and the hybrid scheme follow.
- Textbook RSA is malleable ($c_1 c_2 = (m_1 m_2)^e$) and deterministic (same p, same c: dictionary attack on small message spaces such as a 2-digit password); real systems use OAEP padding.

From the TP: OpenSSL Mission 3: keys, weak RSA, hybrid encryption

`openssl genrsa -out mySmall.pem 512` creates an *unencrypted* private key ("is the private key encrypted? no; is it safely stored? no, anyone who reads the file owns the key"); `openssl rsa -in key.pem -text -noout` shows n, e, d, p, q and CRT values; `-pubout` extracts the public key (fine in plaintext: it is public); `-aes256` when generating encrypts the private key file with AES-256-CBC under a passphrase; `pkeyutl -encrypt -pubin` encrypts a short file and fails on a large one (RSA can only encrypt data smaller than the modulus minus padding). Hybrid scheme: generate a random AES key and IV, encrypt the long message with AES-256-CBC, encrypt the AES key with the partner's RSA public key, send ciphertext + encrypted key + IV; the partner decrypts the key with their private key, then the message. Why: AES is fast and handles any length; RSA handles only a key-sized secret but needs no pre-shared key.

Hard questions, answered

Q. Generate an RSA key pair with $p = 7$, $q = 13$, $e = 5$; encrypt $m = 2$.
A. $n = 91$, $\varphi = 72$, $\gcd(5, 72) = 1$; $d = 29$ ($5 \times 29 = 145 = 2 \times 72 + 1$). $c = 2^5 \bmod 91 = 32$. Check: $32^{29} \bmod 91 = 2$.

Q. Why is choosing p and q "similar in magnitude" recommended, and why "distinct"?
A. If one prime is small, trial division finds it; if they are too close, Fermat's method ($n = a^2 - b^2$) finds them; if $p = q$, $n = p^2$ is a perfect square and φ is wrong.

Q. Eve sees Bob's public key (n, e) and ciphertext c of a 4-digit PIN. Attack without factoring?
A. Encrypt all 10^4 PINs with (n, e) and compare with c: textbook RSA is deterministic. Defence: randomised padding (OAEP).

Q. Why can a 50-bit RSA modulus be broken in the TP by a loop while 2048-bit cannot, if "both are RSA"?
A. Security is quantitative: factoring cost grows super-polynomially in the bit length; $\sqrt{2^{50}} = 2^{25}$ trial divisions is trivial, 2^{1024} is not, and even the best algorithms (GNFS) stop around 800–900 bits today (RSA-250 in 2020).

5.3 Man in the middle: confidentiality is not enough

Core (what the slides say)

With symmetric or asymmetric crypto, Alice and Bob get confidentiality against an eavesdropper. But a **man in the middle** intercepts and alters: "Hi Bob, it's Alice, let's agree on key k_1" / "...key k_2"; Eve hears all and can change anything ("Transfer \$10" → "Transfer \$100"). Can Bob detect the change? Can he know it came from Alice? *Need to authenticate endpoints.*

Hard questions, answered

Q. Does encrypting the DH exchange with Bob's public key stop the MITM?
A. Only if Alice already has Bob's *authentic* public key; if Eve can substitute her public key for Bob's, she decrypts, re-encrypts and relays. The root issue is authenticity of keys, hence signatures and certificates.

5.4 Hash functions

Core (what the slides say)

A **hash function** computes from input data a fingerprint used to quickly identify the initial data; used to verify integrity. Cryptographic hash: easy to compute $H(x)$; fixed-length output for variable-length input; **preimage resistance** (given y, hard to find x with $H(x) = y$); **second-preimage resistance** (given x, hard to find $x' \neq x$ with $H(x') = H(x)$); **collision resistance** (hard to find *any* distinct m_1, m_2 with equal digests). Collisions must exist (finite output space); the concern is the work to find a useful one. The mapping is lossy (input cannot be reconstructed); a tiny change produces a very different digest. Preimage resistance "is not the same as hiding weak passwords". Integrity example: m = "Transfer \$10", $h = H(m)$; Eve changes to m': $H(m') \neq H(m)$. Common functions: MD5, SHA-1, SHA-2 (SHA-224/256/384/512). **Password hashing**: store H(password) instead of plaintext (Alice → `alice1989` exposed if the DB is stolen); the server hashes the entered password and compares; a stolen digest is harder to use, but weak passwords remain vulnerable to **dictionary attacks** (precompute hashes of common passwords; same password ⇒ same unsalted digest; one dictionary reused across accounts). **Salting**: a different random salt per account, combined with the password before hashing, stored alongside the verifier; the salt need not be secret; unique salts prevent one dictionary matching all users, the adversary tests guesses separately per salt; salts do not make a weak password strong. Other uses: digest a message before signing; tamper detection against a trusted digest.

Go deeper: details she can ask about

- Ordering of strength: collision resistance implies second-preimage resistance (if you can find a second preimage you have a collision), but neither implies preimage resistance in general. Birthday bound: collisions in $\approx 2^{n/2}$ work for an n-bit hash (2^{64} for a 128-bit MD5), preimages $\approx 2^n$.
- MD5 and SHA-1 are *broken for collisions* (practical collisions exist), still fine as checksums against accidental corruption; SHA-2 is current.
- "Not the same as hiding weak passwords": preimage resistance assumes an *unknown, high-entropy* input; a password from a small dictionary is found by enumeration no matter how good the hash. Hence salt (kills precomputation/sharing) and slow hashes (PBKDF2, bcrypt, scrypt, Argon2) which raise the per-guess cost; the TP's `-pbkdf2` flag is exactly a password-based key derivation.
- Salt is stored next to the hash and need not be secret because its job is to make each user's hash computation *different*, not to be unknown.
- Quiz-style trap: a hash alone gives integrity against *accidental* change, not against an *adversary* who can replace both.

Hard questions, answered

Q. Two users choose the same password. Unsalted vs salted: what does the attacker see in the database, and what does that enable?
A. Unsalted: identical digests, so cracking one cracks both and reveals shared passwords; precomputed tables apply to every account. Salted: different digests; each account requires its own guessing campaign.

Q. Why is "MD5 is broken" true for signatures but less relevant for file download checksums?
A. Broken means collisions are findable by whoever crafts both inputs (forging two documents with the same digest). A checksum published by a trusted site against an attacker who cannot change the published digest still needs a *second preimage*, which remains hard; but if the attacker can influence the file before hashing, collisions matter.

Q. Alice sends $(m, H(m))$ over an insecure channel. Does Bob detect modification?
A. No. Eve replaces the pair by $(m', H(m'))$ and the check passes; H has no secret. Integrity against an adversary needs a key: MAC (shared secret) or signature (private key).

Q. Explain why hashing the password on the *client* before sending does not remove the need for server-side hashing.
A. The client-side hash becomes the effective password: whoever steals the database can replay it. The server must still hash (with salt) what it receives.

5.5 Authentication: MACs and digital signatures

Core (what the slides say)

The authenticity problem: who sent this message? Solution: the sender includes data only they could have created. Asymmetric crypto can authenticate: Bob encrypts with his *private* key; anyone with his public key can decrypt, proving Bob produced it. Signing long messages directly is slow and risky (chosen-message attacks on the private key), so sign a *hash*: signature size independent of message size, security maintained. Two approaches: **MAC** (symmetric: shared secret key; sender computes $t = \mathrm{MAC}(K, m)$, receiver recomputes and compares; Eve without K cannot forge a valid tag) and **digital signature** (asymmetric: sender's private key, receiver verifies with the sender's public key). Confidentiality and authenticity are separate: confidentiality without authenticity = secrets sent anonymously; authenticity without confidentiality = public

info from a trusted source; both = secure communication. MAC implementations: last block of CBC encryption of the message (sender needed key and message together; expensive), or hash-based schemes (hash + symmetric encryption; hash of message concatenated with a secret value), requiring *cryptographic* hash functions. MAC provides integrity + shared-key authentication; *no* confidentiality and *no* non-repudiation. Digital signature (NIST FIPS 186-4): origin authentication, data integrity, signatory **non-repudiation** (you cannot deny you sent it); algorithms RSA signature, DSA, ECDSA; advantage over MAC: no pre-shared key.

Go deeper: details she can ask about

- Why a MAC cannot give non-repudiation: both parties hold K, so Bob could have produced the tag himself; a judge cannot tell. A signature uses a key only Alice holds.
- "Encrypt with the private key" is the textbook picture; real signatures are `Sign(priv, H(m))` with padding; verification recomputes $H(m)$ and checks the signature. The hash makes the scheme size-independent and blocks the "carefully chosen messages" attack on raw RSA.
- Hash-then-sign needs collision resistance: if Eve finds m_1, m_2 with the same hash and gets m_1 signed, the signature also validates m_2 (the MD5 certificate forgery of 2008 is the canonical case).
- The three-way separation (confidentiality / integrity+authentication / non-repudiation) with the tool for each: encryption / MAC or signature / signature only.
- Figure 2.5 variants: (a) hash then encrypt the hash with a shared key; (c) hash message plus secret value; (b) hash then encrypt with the private key = signature.

Hard questions, answered

Q. Fill the table: for encryption, MAC, signature, give C / I / Auth / NR.
A. Encryption: C yes, I no, Auth no, NR no. MAC: C no, I yes, Auth yes (shared-key), NR no. Signature: C no, I yes, Auth yes, NR yes.

Q. Alice wants to send Bob a confidential, authenticated contract he cannot later deny receiving from her. Which primitives, in which order?
A. Sign with Alice's private key (authenticity + non-repudiation), then encrypt for Bob (hybrid: AES for the content, Bob's public key for the AES key). Signing first binds Alice to the plaintext; encrypting afterwards keeps it confidential.

Q. Why does a MAC need a cryptographic hash rather than a hash-table hash?
A. A table hash minimises collisions on average but is easy to invert and to collide deliberately; a MAC must resist forgery by an adversary choosing inputs, so it needs preimage/collision resistance and key-dependence.

Q. Is `MAC = H(K || m)` a safe construction?
A. Not for Merkle–Damgård hashes (MD5, SHA-1, SHA-256): length-extension lets Eve compute $H(K\|m\|\text{pad}\|m')$ without K. HMAC or SHA-3 avoid it; the slides keep this at the "various schemes" level.

5.6 Certificates, chain of trust, PKI

Core (what the slides say)

The recursive problem: to validate a message signed by Alice we need her public key; Alice sends it as a signed message, but validating *that* needs her key: she cannot remotely prove a key is hers on her own. Solution: someone else (Bob) signs a message containing Alice's key; then Clara signs Bob's; then Don signs Clara's; base case: Don's key "got shipped to us just for this; we trust it implicitly." A **certificate**: a message containing someone's identity and public key, signed by

someone else; each signer is a **certificate authority** (CA); the implicitly trusted one is a **root CA**; together they form a **chain of trust** (Alice ← Bob "CA" ← Clara "CA" ← Don "Root CA"). Getting a certificate: (1) generate a key pair, keep the private key secret forever; (2) present the public key to the CA in a **certificate signing request** (CSR: everything except the signature) with proof of identity; (3) the CA verifies and signs the CSR, producing the certificate; anyone who trusts your CA (directly or recursively) trusts the key is yours. Format: X.509 (RFC 5280). **PKI**: "the set of hardware, software, people, policies and procedures needed to create, manage, store, distribute and revoke digital certificates based on asymmetric cryptography" (RFC 4949); includes a **trust store** (list of CAs and their public keys), chosen by the OS or app vendor (a major decision); organisations add a private root CA to sign internally; if malware can add a root CA it can sign *any* certificate and mount MITM attacks; some security software does this to inspect encrypted traffic (the slide calls it "stupid and dangerous").

Go deeper: details she can ask about

- The base case is *trust by distribution*, not by cryptography: root keys arrive with the OS-/browser. Everything above rests on that out-of-band step, which is why the trust store is "a major decision".
- A CSR contains the public key and identity but not the private key; the CA never sees the private key. The signature on a certificate is the CA's, made with the CA's private key over the certificate contents (which include the subject's public key).
- Revocation is in the PKI definition (RFC 4949) and is the hard part in practice (CRLs, OCSP) because a certificate cannot be "unsent".
- TLS interception appliances work exactly by adding a private root CA to all machines; the slide's objection: a single compromise of that CA breaks confidentiality for everyone behind it.

Hard questions, answered

Q. Why can't Alice just sign her own public key to prove it is hers?
A. Verifying her signature requires her public key, the very thing in question; self-signature proves only that whoever made the key pair made the signature. Trust must enter from outside (a CA or out-of-band delivery).

Q. Walk through what the browser checks when it receives `www.bank.ma`'s certificate.
A. Signature of the certificate by the intermediate CA's key (found in the chain), the intermediate's certificate signed by a root in the trust store, validity dates, that the name matches, that none is revoked. Then it uses the server's public key for key exchange; a failure anywhere means "not Bob".

Q. Malware adds a root CA to a laptop. What exactly can the attacker now do, and what remains protected?
A. Issue a certificate for any site the browser will accept, so MITM every HTTPS connection from that laptop (read and alter traffic). Other users' laptops are unaffected; sites using certificate pinning or users checking fingerprints may notice.

Q. Difference between a CA signing a CSR and a CA generating your key for you.
A. With a CSR you keep the private key; the CA only attests the public key. If the CA generates the pair it has your private key, destroying non-repudiation and confidentiality guarantees.

5.7 Steganography

Core (what the slides say)

Steganography: data hidden in plain sight; the intruder does not learn of the *existence* of the secret message (cryptography hides the *meaning*; both hide messages in a medium). Simmons (1983) brought it into computer science with the **Prisoners' Problem**: Alice and Bob communicate through the warden; *stego objects* are the communications, the *payload* is the hidden information; a *passive* warden only eavesdrops, an *active* warden tampers with stego objects. First technique in ancient Greece around 440 BC. **Null cipher**: take the n-th letter of each word ("Fishing freshwater bends…" third letters → "Send Lawyers, Guns, and Money"; the WWI telegram: first letters of the first paragraph and second letters of the second → "PERSHING SAILS FROM N.Y. JUNE 1"). Types: text, image, audio, video, network, email steganography. "The sheer size of the Internet" makes hiding effective. **LSB steganography**: overwrite the bit with the lowest arithmetic value of each pixel/sample with payload bits.

Go deeper: details she can ask about

- Crypto vs stego in one line: crypto = "you see a message but cannot read it"; stego = "you do not see a message". They compose: encrypt the payload, then hide it.
- LSB: changing the least significant bit of an 8-bit grey value alters it by at most 1/255, invisible; capacity is 1 bit per byte (an image of N bytes hides $N/8$ bytes); detection (steganalysis) looks at LSB statistics; an *active* warden defeats LSB simply by re-compressing or adding noise.
- The warden model is the threat model of stego: passive warden = detection problem; active warden = robustness problem.

Hard questions, answered

Q. Hide the byte `01000001` ('A') in the pixels `10110010, 10110011, 10110010, 10110011, 10110010, 10110011, 10110010, 10110011`.
A. Replace each LSB: `10110010, 10110011, 10110010, 10110010, 10110010, 10110010, 10110010, 10110011`. Each value changes by at most one unit.

Q. Which warden type defeats LSB stego trivially, and how?
A. The active warden: re-encoding the image (JPEG compression, slight blur, re-sampling) destroys the LSB plane while leaving the picture intact for the receiver. A passive warden must instead detect statistical anomalies.

Q. Why combine encryption with steganography?
A. If the warden finds the payload, encryption keeps its meaning hidden; stego alone offers no confidentiality once detected.

A Quiz 1 (Web Privacy, "The Alice Privacy Investigation"), solved

Multi-select: every correct answer must be ticked. The reasoning is the part she grades in oral follow-ups.

Q1. Reading a medical record without change: **A** confidentiality.
Q2. Changing a lab result while it stays accessible: **B** integrity.
Q3. DoS on the portal: **C** availability.
Q4. `aid=A72; Domain=example.test; Path=/; Secure; HttpOnly; SameSite=Lax`: **A, C**. A: same site, HTTPS. B: `http` violates Secure. C: cross-site top-level GET navigation is allowed by Lax. D: cross-site embedded subresource blocked by Lax.

Q5. Non-HttpOnly first-party `prefs`, analytics script from `metrics.test`, iframe from `tracker.test`: **A, D**. The script runs in the shop's origin (reads `prefs`); the iframe is another origin (cannot); cookies are never sent to a domain they do not belong to (C false); the script can send what it read explicitly (D).

Q6. Pixel from `tracker.test` with `tid; SameSite=None; Secure` on two publishers, 3P cookies allowed: **A, D**. Publisher JS cannot read a tracker cookie (B false); SOP does not control cookie attachment (C false).

Q7. 3P cookies blocked, same analytics script, `aid=N31`/`aid=S84`: **A, D**. B: no automatic sending of first-party cookies to `metrics.test`. C: same script, different cookie jars.

Q8. Sync exchange `tracker-a` → 302 → `tracker-b/sync?aid=A42` with `bid=B71`: **A, C**. B: A42 arrives as a URL parameter, not a cookie. D: different origins are the whole point.

Q9. RTB, DSP-X loses the auction: **A, B**. All DSPs receive the bid request (C false); the pseudonymous ID is stable and linkable (D false).

Q10. Tests 2 and 3: **A, C**. Test 2 consistent with localStorage respawn; it does not *prove* fingerprinting (B false); Test 3 consistent with ETag respawn; the profile stayed 3, so deletion did not break the link (D false).

Q11. Tests 4 and 5: **B, D**. Fingerprint 88 identifies a browser, not a named person (A false); new IDs 5678 show the stores *were* cleared (C false); shared fingerprints give false positives (D true).

Q12. Sufficient against both respawns: **A** (cookies, local storage and cache together).

Q13. Passive data before JS: **A**.

Q14. Canvas fingerprinting: **B** (render predefined text/shapes, read pixels, hash; output varies across configurations).

Q15. Entropy of OS with 50/25/25: **C**, 1.5 bits ($0.5 \cdot 1 + 0.25 \cdot 2 + 0.25 \cdot 2$).

Q16. Cookie syncing design: see the answer in §2.7 (Tracker 1 pixel → reads `1234` → `302` to `tracker2/sync?t1=1234` → browser sends Tracker 2's cookie `5678` → Tracker 2 stores (5678 ↔ 1234) → optional redirect back with `t2=5678`). In the drawing, show: the browser in the middle, both trackers' cookie jars, the redirect arrow carrying the ID in the URL, and the mapping table on Tracker 2's side.

Do not confuse

Patterns in her quiz: (i) every cookie question hinges on *which domain the request is to* and *whether the request is top-level*; (ii) "consistent with" ≠ "proves"; (iii) identifiers in URLs are not cookies; (iv) "third-party cookies blocked" does not block first-party cookies set by third-party scripts; (v) fingerprints identify devices, with false positives. Expect the midterm to reuse these five moves on new scenarios (DNS/CNAME, Topics, partitioned cookies).

B What each TP teaches that can come back on the exam

1. **TP1 Clickjacking.** Iframe + `opacity: 0` + `z-index` + aligned buttons; a transparent frame still receives clicks; the attack needs an authenticated victim. Defences are set by the *framed* site: `X-Frame-Options: DENY | SAMEORIGIN` (ALLOW-FROM obsolete), CSP `frame-ancestors 'none' | 'self' | <origins>`; only CSP can whitelist a specific trusted embedding application; `SameSite` session cookies reduce impact. See §2.8.
2. **TP Stateful tracking (Flask).** Predict → test → inspect → explain. Custom response header `X-Lab-Message`; cookie set with `set_cookie`; restarting the browser removes a session cookie, keeps a persistent one; the Windows `hosts` file maps `lab.test` to `127.0.0.1` so that different hostnames exercise first/third-party rules. Challenge 1: two publishers embed an iframe from `tracker-one.test`; the tracker's cookie is the same on both, the query parameter tells it which publisher/page; the server reconstructs a chronological profile. Challenge 2: `analytics.js` from `analytics.test` runs in the publisher's context, sets *first-party* cookies, so each publisher gets a different identifier and the analytics server does *not* automatically receive them; it gets what the script sends; result: separate profiles per site unless linked otherwise. Optional: cookie syncing by redirect (A123 ↔ B789).

3. **TP Fingerprinting.** Passive (IP, User-Agent, Accept-Language, Accept-Encoding, Accept, Referer, Sec-CH-* client hints, Sec-Fetch-* and other security headers); active (`navigator.language`, `.languages`, `.userAgent`, `.hardwareConcurrency`, `.deviceMemory`; `screen.width`, `.height`, `.colorDepth`; `window.innerWidth`, `.devicePixelRatio`; `Intl` time zone; `document.referrer`); behavioural (typing time, speed = characters or words per unit time, Backspace count, key-hold, inter-key interval, pauses); SHA-256 of features in a fixed order via `crypto.subtle.digest()` as a stable short identifier; private mode and cookie blocking change *nothing* in the fingerprint, only tools that spoof or randomise do, and those can be detected (FP-Scanner) or themselves be rare.
4. **TP Breaking the Classics.** Caesar brute force (26 keys); substitution needs frequency analysis (26! keys); Vigenère with key length $L = L$ interleaved Caesar ciphers (`text[i::L]`), each solved by "most frequent → E", shift → key letter; keep your code. See §4.3.
5. **TP AES.** AddRoundKey = XOR (self-inverse); SubBytes via S-box (`00`→`63`, `03`→`7B`, `71`→`A3`); ShiftRows left by row index; MixColumns in GF(2^8) with `xtime` and `1B`; final round without MixColumns; 11 round keys; decryption order; the no-diffusion two-round byte cipher broken with 2^{16} guesses; the message is that confusion without diffusion is attackable byte by byte. See §4.7.
6. **TP OpenSSL.** RSA by hand ($p = 5$, $q = 11$, $e = 3$, $d = 27$, $C = 23 \to m = 12$); `Salted__` + 8-byte salt header; ECB reveals image structure (equal blocks), CBC does not; `openssl enc -d -aes-256-ecb -pbkdf2 -p`; `genrsa`, `rsa -text -noout`, `-pubout`, `pkeyutl -encrypt -pubin`; an unencrypted private-key file is a liability, a public key may be plaintext; a 50-bit modulus is factored by trial division, 512-bit is not; RSA cannot encrypt large files, hence hybrid encryption (AES key + IV, key wrapped with RSA). See §4.5, §5.2.

Do not confuse

Her exam questions mix layers: a crypto question will carry a web detail ("the IV is sent in the cookie"), a privacy question a crypto detail ("the fingerprint is SHA-256 hashed, is it anonymous?"). Answer with the mechanism (what the browser/algorithm actually does), then the property (C/I/A, uniqueness/linkability), then the limitation.